

NFL Officiating Patterns

Olivia Darby, Chase Foster, Ryan Wenstrom

```
In [1]: # Import statements
import numpy as np
import seaborn as sns

import pandas as pd
pd.options.display.float_format = '{:,.2f}'.format

import matplotlib.pyplot as plt
%matplotlib inline

import plotly.express as px
```

1. Introduction & Guiding Questions

You may have seen the commercials: “Special profit boost!” “Same-game parlay odds!” “Bet live during the game!” In today’s professional sports broadcasts, betting odds scroll across screens as often as scores, and every call sparks debate. What used to be a sideline hobby is now woven into the fabric of the NFL. The 2018 Supreme Court ruling in *Murphy v. National Collegiate Athletic Association* brought rise to this development by removing the federal prohibition on sports betting, and as of 2025, 38 states offer legal sports betting in some form. In the last year alone, Americans wagered \$150 billion on sports, the majority of that activity happening online. With so much money riding on every snap, the pressure on officiating has reached a new level. Fans frequently allege bias, inconsistency, or even manipulation in officiating — claims that are highly consequential for public trust and the integrity of the sport. This exploratory project uses play-by-play and game-level data from the 2015-2024 NFL seasons to see if the new era of legalized sports has had an impact on the way the game is called. The goal of this project is to describe how penalties, officiating crews and replay decisions have changed over time and whether these changes align with the shift into the modern sports betting era. For our analysis, we will consider the pre-betting era to be from 2015-2019 as the majority of US states didn’t legalize sports betting until after the 2018-2019 NFL season ended.

Our primary datasets are ‘play-by-play’ (pbp) and ‘games’ (game_data), accessed through the open-source package nflreadpy: <https://github.com/nflverse/nflreadpy>

This package retrieves public data from the NFLverse data repository, which aggregates detailed play-by-play statistics across multiple seasons. The ‘play-by-play’ dataset includes detailed play-level records (penalty flags, EPA, WPA, play type, review/challenge markers, etc.). The ‘games’ dataset includes game-level metadata (pre-game moneyline/spread/total,

kickoff time, location, result, final score differential). These will be merged and filtered to conduct our analysis.

The following main questions will guide the remainder of our report:

- 1.) How have overall NFL penalty trends (rates, types, and yardage) changed between the pre-betting and post-betting era?
- 2.) Do certain officiating crews differ in their frequency of calling penalties, and have these tendencies changed between the pre-betting and post-betting era?
- 3.) How have replay review outcomes, specifically reversals versus upheld calls, shifted between the pre-betting and post-betting era?

2. Data Preparation & Cleaning

2.1 Loading the Data

Play-By-Play Data (2015-2024)

We began by loading play-by-play data from the 2015 season to the 2024 season using `nflreadpy`. This contains most of the play-specific information we will need for our analysis. We chose this time-frame in order to observe more accurate temporal patterns within the data. Additionally, the federal ban on sports betting was struck down in May of 2018 (with sports betting being legal in 38 states as of 2025), so this span will contain enough data points to observe officiating changes (if any) both before and after this occurred.

Side-note: `nflreadpy` imports the data as a Polars dataframe, so we converted all to Pandas before moving forward.

```
In [2]: import nflreadpy as nfl

# Load season play-by-play data
plays = nfl.load_pbp([2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024])
```

```
In [3]: # Converting to pandas
plays = plays.to_pandas()
```

```
In [4]: plays.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 483605 entries, 0 to 483604
Columns: 372 entries, play_id to pass_oe
dtypes: float64(199), int32(8), object(165)
memory usage: 1.3+ GB
```

```
In [5]: plays.head()
```

Out[5]:

		play_id	game_id	old_game_id	home_team	away_team	season_type	week	pr
0	1.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1	
1	36.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1	
2	51.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1	
3	75.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1	
4	96.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1	

5 rows × 372 columns



Game Data (2015-2024)

Game data (obtained by 'nfl.load_schedules()') provides game context and information, including the result, betting lines, and, most importantly, the lead official (referee). This was merged with 'plays' to obtain a comprehensive dataset for analysis.

```
In [6]: # Loading game contextual data
games = nfl.load_schedules([2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024])
```

```
In [7]: # Converting to pandas
games = games.to_pandas()
```

```
In [8]: games.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2743 entries, 0 to 2742
Data columns (total 46 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   game_id               2743 non-null   object
 1   season                2743 non-null   int32
 2   game_type             2743 non-null   object
 3   week                  2743 non-null   int32
 4   gameday               2743 non-null   object
 5   weekday               2743 non-null   object
 6   gametime              2743 non-null   object
 7   away_team             2743 non-null   object
 8   away_score            2743 non-null   int32
 9   home_team             2743 non-null   object
10   home_score            2743 non-null   int32
11   location              2743 non-null   object
12   result                2743 non-null   int32
13   total                 2743 non-null   int32
14   overtime              2743 non-null   int32
15   old_game_id           2743 non-null   object
16   gsis                  2743 non-null   int32
17   nfl_detail_id         272 non-null    object
18   pfr                   2743 non-null   object
19   pff                   1889 non-null   float64
20   espn                  2743 non-null   object
21   ftn                   1376 non-null   float64
22   away_rest             2743 non-null   int32
23   home_rest             2743 non-null   int32
24   away_moneyline        2742 non-null   float64
25   home_moneyline        2742 non-null   float64
26   spread_line           2743 non-null   float64
27   away_spread_odds      2742 non-null   float64
28   home_spread_odds      2742 non-null   float64
29   total_line            2743 non-null   float64
30   under_odds            2742 non-null   float64
31   over_odds             2742 non-null   float64
32   div_game              2743 non-null   int32
33   roof                  2743 non-null   object
34   surface               2743 non-null   object
35   temp                  1806 non-null   float64
36   wind                  1806 non-null   float64
37   away_qb_id            2743 non-null   object
38   home_qb_id            2743 non-null   object
39   away_qb_name          2743 non-null   object
40   home_qb_name          2743 non-null   object
41   away_coach            2743 non-null   object
42   home_coach            2743 non-null   object
43   referee              2742 non-null   object
44   stadium_id            2743 non-null   object
45   stadium               2743 non-null   object
dtypes: float64(12), int32(11), object(23)
memory usage: 868.0+ KB

```

```
In [9]: games.head()
```

Out[9]:

	game_id	season	game_type	week	gameday	weekday	gametime	away_team
0	2015_01_PIT_NE	2015	REG	1	2015-09-10	Thursday	20:30	PIT
1	2015_01_IND_BUF	2015	REG	1	2015-09-13	Sunday	13:00	IND
2	2015_01_GB_CHI	2015	REG	1	2015-09-13	Sunday	13:00	GB
3	2015_01_KC_HOU	2015	REG	1	2015-09-13	Sunday	13:00	KC
4	2015_01_CAR_JAX	2015	REG	1	2015-09-13	Sunday	13:00	CAR

5 rows × 46 columns



Supplemental Datasets

The nflreadpy package also allows us to import the secondary 'teams' dataset, which includes team IDs, logos, and color codes. This will aid in our final visualizations.

```
In [10]: # Team colors, Logos, and IDs for visualizations
teams = nfl.load_teams()
```

```
In [11]: # Converting to pandas
teams = teams.to_pandas()
```

```
In [12]: teams.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36 entries, 0 to 35
Data columns (total 16 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   team_abbr              36 non-null    object  
 1   team_name              36 non-null    object  
 2   team_id                36 non-null    int32   
 3   team_nick              36 non-null    object  
 4   team_conf              36 non-null    object  
 5   team_division          36 non-null    object  
 6   team_color             36 non-null    object  
 7   team_color2            36 non-null    object  
 8   team_color3            35 non-null    object  
 9   team_color4            35 non-null    object  
10   team_logo_wikipedia    36 non-null    object  
11   team_logo_espn         36 non-null    object  
12   team_wordmark          36 non-null    object  
13   team_conference_logo   36 non-null    object  
14   team_league_logo       36 non-null    object  
15   team_logo_squared      36 non-null    object  
dtypes: int32(1), object(15)
memory usage: 4.5+ KB

```

In [13]: `teams.head()`

Out[13]:

	team_abbr	team_name	team_id	team_nick	team_conf	team_division	team_color	team
0	ARI	Arizona Cardinals	3800	Cardinals	NFC	NFC West	#97233F	
1	ATL	Atlanta Falcons	200	Falcons	NFC	NFC South	#A71930	
2	BAL	Baltimore Ravens	325	Ravens	AFC	AFC North	#241773	
3	BUF	Buffalo Bills	610	Bills	AFC	AFC East	#00338D	
4	CAR	Carolina Panthers	750	Panthers	NFC	NFC South	#0085CA	

2.2 Merging Plays and Games

To merge the 'plays' and 'games' datasets, we used 'game_id', 'season', 'week', 'home_team', 'away_team', 'result', 'home_score', 'away_score', and 'spread_line' as the key variables, which exist in both datasets. There are other shared variables between the two, but we did not specify these in the merge as we will be removing them once we create the subset in the next section.

However, before merging, we will need to correct the team abbreviations used for 'home_team' and 'away_team'. Several teams relocated to new cities since 2015, thus their abbreviations have changed (e.g. The Los Angeles Rams moved from St. Louis in 2016, so their abbreviation went from STL to LAR). These old abbreviations are used in 'games', but 'plays' applies the current ones for standardization (even in years before the move). This is shown below:

```
In [14]: plays['home_team'].value_counts()
```

```
Out[14]: home_team
KC      17491
PHI     16406
BUF     16015
NE      15782
TB      15718
NO      15523
HOU     15496
BAL     15419
DAL     15372
LA      15354
SF      15302
PIT     15139
ATL     15074
DEN     15064
DET     15047
CIN     15039
GB      15025
MIN     14977
SEA     14959
CAR     14894
ARI     14780
JAX     14732
TEN     14694
WAS     14649
NYG     14558
LV      14547
CLE     14522
NYJ     14520
IND     14500
MIA     14429
LAC     14365
CHI     14213
Name: count, dtype: int64
```

```
In [15]: games['home_team'].value_counts()
```

```
Out[15]: home_team
        KC      100
        PHI      92
        BUF      91
        NE       90
        TB       88
        HOU      88
        SF       88
        NO       88
        BAL      87
        GB       87
        DAL      86
        ATL      85
        PIT      85
        DEN      85
        MIN      85
        DET      85
        CIN      85
        TEN      84
        SEA      84
        CAR      84
        WAS      84
        JAX      84
        ARI      83
        CHI      83
        CLE      82
        NYG      82
        IND      82
        NYJ      82
        MIA      82
        LA       80
        LAC      66
        LV       42
        OAK      40
        SD       16
        STL       8
Name: count, dtype: int64
```

So, we will have to apply a new team abbreviation mapping so as not to interfere with the merge:

```
In [16]: # Fixing team abbreviations
team_abbr_mapping = {
    'OAK': 'LV',
    'SD': 'LAC',
    'STL': 'LAR',
    'LA' : 'LAR',
}

# Replace old abbreviations in relevant columns
for col in ['home_team', 'away_team', 'posteam', 'defteam', 'penalty_team']:
    plays[col] = plays[col].replace(team_abbr_mapping)

for col in ['home_team', 'away_team']:
    games[col] = games[col].replace(team_abbr_mapping)
```

```
In [17]: # verifying team abbreviations in plays and games
plays['home_team'].value_counts()
```

```
Out[17]: home_team
KC      17491
PHI     16406
BUF     16015
NE      15782
TB      15718
NO      15523
HOU     15496
BAL     15419
DAL     15372
LAR     15354
SF      15302
PIT     15139
ATL     15074
DEN     15064
DET     15047
CIN     15039
GB      15025
MIN     14977
SEA     14959
CAR     14894
ARI     14780
JAX     14732
TEN     14694
WAS     14649
NYG     14558
LV      14547
CLE     14522
NYJ     14520
IND     14500
MIA     14429
LAC     14365
CHI     14213
Name: count, dtype: int64
```

```
In [18]: games['home_team'].value_counts()
```

```
Out[18]: home_team
KC      100
PHI      92
BUF      91
NE       90
SF       88
LAR      88
NO       88
TB       88
HOU      88
BAL      87
GB       87
DAL      86
CIN      85
PIT      85
MIN      85
DET      85
ATL      85
DEN      85
CAR      84
WAS      84
TEN      84
SEA      84
JAX      84
CHI      83
ARI      83
NYJ      82
CLE      82
LV       82
NYG      82
LAC      82
IND      82
MIA      82
Name: count, dtype: int64
```

Now, we can move forward with the merge:

```
In [19]: # Merging plays and games
plays_games = plays.merge(
    games,
    on = ['game_id', 'home_team', 'away_team', 'season', 'week',
          'result', 'home_score', 'away_score', 'spread_line'],
    how = 'left'
)
```

```
In [20]: plays_games.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 483605 entries, 0 to 483604
Columns: 409 entries, play_id to stadium_y
dtypes: float64(210), int32(14), object(185)
memory usage: 1.4+ GB
```

```
In [21]: plays_games.head()
```

Out[21]:

		play_id	game_id	old_game_id_x	home_team	away_team	season_type	week
0	1.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1
1	36.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1
2	51.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1
3	75.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1
4	96.00	2015_01_BAL_DEN	2015091309		DEN	BAL	REG	1

5 rows × 409 columns



2.3 Creating the Subset

There are 409 variables in the resulting 'plays_games' dataset. For our analysis, we have determined that we will only need the following:

Game and Play Context

- 'game_id': game identifier (string)
- 'play_id': play identifier; unique for a single play (when used along with game_id) (numeric)
- 'referee': name of the game's referee (head official) (string)
- 'home_team': abbreviation; represents the home team in a given game (string)
- 'away_team': abbreviation; represents the away team in a given game (string)
- 'season_type': denotes whether a game occurred within the regular season or post season (string)
- 'season': indicating the season in which the game was played (numeric)
- 'posteam': indicates the team with possession of the ball during a given play (offense) (string)
- 'defteam': indicates the team on defense (string)
- 'down': represents the down for the given play (numeric)

- 'ydstogo': indicates distance (in yards) from either the first down marker or the endzone in goal down situations (numeric)
- 'play_type': indicates the type of play: pass (includes sacks), run (includes scrambles), punt, field_goal, kickoff, extra_point, qb_kneel, qb_spike, no_play (timeouts and penalties) (string)

Play/Game Results and Penalties

- 'epa': represents the expected points added (EPA) by the team in possession for the given play (numeric)
- 'wp': estimated win probability for the team in possession given the current situation at the start of the given play (numeric)
- 'wpa': win probability added (WPA) for the team in possession (after the play) (numeric)
- 'yards_gained': yards gained (or lost) by the possessing team, excluding yards gained via fumble recoveries and laterals (numeric)
- 'penalty': binary indicator of whether or not a penalty occurred (numeric)
- 'penalty_team': indicates which team committed the penalty (string)
- 'penalty_yards': yards gained (or lost) by the team in possession from the penalty (numeric)
- 'penalty_type': indicates the penalty type of the first penalty in the given play (string)
- 'replay_or_challenge': binary indicator of whether or not a replay or challenge occurred on the play (numeric)
- 'replay_or_challenge_result': the result of the replay/challenge (string)
- 'result': the number of points the home team scored minus the number of points the visiting team scored (numeric)

```
In [22]: # Specifying which columns to keep
keep_cols = [
    'game_id', 'play_id', 'home_team', 'away_team', 'season_type',
    'season', 'posteam', 'defteam', 'ydstogo', 'down', 'play_type',
    'score_differential', 'epa', 'wp', 'wpa', 'penalty', 'penalty_type',
    'penalty_yards', 'penalty_team', 'yards_gained',
    'replay_or_challenge', 'replay_or_challenge_result',
    'result', 'referee'
]

# Creating the subset
plays_games_subset = plays_games[keep_cols].copy()
```

```
In [23]: plays_games_subset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 483605 entries, 0 to 483604
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   game_id               483605 non-null object
1   play_id               483605 non-null float64
2   home_team             483605 non-null object
3   away_team             483605 non-null object
4   season_type           483605 non-null object
5   season                483605 non-null int32
6   posteam               457974 non-null object
7   defteam               457974 non-null object
8   ydstogo               483605 non-null float64
9   down                  407053 non-null float64
10  play_type             469418 non-null object
11  score_differential    457944 non-null float64
12  epa                   478079 non-null float64
13  wp                    480824 non-null float64
14  wpa                   476455 non-null float64
15  penalty               468937 non-null float64
16  penalty_type          34190 non-null object
17  penalty_yards         34207 non-null float64
18  penalty_team          34207 non-null object
19  yards_gained          468937 non-null float64
20  replay_or_challenge   483605 non-null float64
21  replay_or_challenge_result 3754 non-null object
22  result                 483605 non-null int32
23  referee               483443 non-null object
dtypes: float64(11), int32(2), object(11)
memory usage: 84.9+ MB
```

2.4 Data Cleaning

Handling Missing Values

To clean the data, we will first check where nulls are present in the info table. The first category to fix are:

- 'penalty_yards', 'penalty_type', 'penalty_team': null when 'penalty' = 0
- 'replay_or_challenge_result': null when 'replay_or_challenge' = 0

These null values are legitimate parts of data, and are to be expected (if a penalty/challenge didn't happen, it makes sense for penalty/challenge-related qualifiers to be null). So, we will set all null values of 'penalty_yards' to be '0' (since it's a numeric variable), and null values of 'penalty_type', 'penalty_team', and 'replay_or_challenge_result' to be 'None' (since these are character variables).

```
In [24]: # Setting NAs to 0 for penalty_yards; 'None' for penalty_type, penalty_team, replay
plays_games_subset['penalty_type'] = plays_games_subset['penalty_type'].fillna('None')
plays_games_subset['penalty_team'] = plays_games_subset['penalty_team'].fillna('None')
```

```
plays_games_subset['penalty_yards'] = plays_games_subset['penalty_yards'].fillna(0)
plays_games_subset['replay_or_challenge_result'] = plays_games_subset['replay_or_ch
```

We will also drop NAs for 'referee', as this variable is needed for our analysis, and filling them with some other placeholder value can negatively affect our comparisons (mixing multiple unknown referees together creates misleading aggregates):

```
In [25]: plays_games_subset['referee'].isna().sum()
```

```
Out[25]: 162
```

```
In [26]: # Dropping NAs for referee
plays_games_subset = plays_games_subset.dropna(subset=['referee'])
```

```
In [27]: plays_games_subset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 483443 entries, 0 to 483604
Data columns (total 24 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   game_id               483443 non-null object
 1   play_id              483443 non-null float64
 2   home_team            483443 non-null object
 3   away_team            483443 non-null object
 4   season_type          483443 non-null object
 5   season               483443 non-null int32
 6   posteam              457821 non-null object
 7   defteam              457821 non-null object
 8   ydstogo              483443 non-null float64
 9   down                 406918 non-null float64
10  play_type             469261 non-null object
11  score_differential    457791 non-null float64
12  epa                   477919 non-null float64
13  wp                    480663 non-null float64
14  wpa                   476296 non-null float64
15  penalty              468781 non-null float64
16  penalty_type         483443 non-null object
17  penalty_yards        483443 non-null float64
18  penalty_team         483443 non-null object
19  yards_gained         468781 non-null float64
20  replay_or_challenge   483443 non-null float64
21  replay_or_challenge_result 483443 non-null object
22  result               483443 non-null int32
23  referee              483443 non-null object
dtypes: float64(11), int32(2), object(11)
memory usage: 88.5+ MB
```

Now, we will drop all null values that don't represent a play. For example, null values of 'down' represent kickoffs, PATs, some penalties, and some special plays, while 'play_type' wouldn't have a reason for this besides being a malformed entry (these account for small percentages of the total entries). For some variables, like 'posteam' and 'defteam', we will first verify whether they belong in the former category by checking to see how the data

classifies the possession team and defending team on kickoffs, as kicking and returning teams on these plays aren't always sorted this way.

```
In [28]: # verifying null posteam/defteam isn't due to kickoffs
plays_games_subset[plays_games_subset['play_type'] == 'kickoff'].info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 28033 entries, 1 to 483600
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   game_id               28033 non-null  object
1   play_id               28033 non-null  float64
2   home_team             28033 non-null  object
3   away_team             28033 non-null  object
4   season_type           28033 non-null  object
5   season                28033 non-null  int32
6   posteam               28033 non-null  object
7   defteam               28033 non-null  object
8   ydstogo               28033 non-null  float64
9   down                  0 non-null      float64
10  play_type             28033 non-null  object
11  score_differential     28033 non-null  float64
12  epa                   28033 non-null  float64
13  wp                    28033 non-null  float64
14  wpa                   27954 non-null  float64
15  penalty               28033 non-null  float64
16  penalty_type          28033 non-null  object
17  penalty_yards         28033 non-null  float64
18  penalty_team          28033 non-null  object
19  yards_gained          28033 non-null  float64
20  replay_or_challenge   28033 non-null  float64
21  replay_or_challenge_result 28033 non-null  object
22  result                28033 non-null  int32
23  referee               28033 non-null  object
dtypes: float64(11), int32(2), object(11)
memory usage: 5.1+ MB
```

This demonstrates that the nulls found in 'posteam' and 'defteam' aren't due to kickoffs, so they can be removed (side note: this also demonstrates how 'down' is NA on kickoff plays --- if there were any non-nulls here, we would have a problem).

```
In [29]: # Dropping NAs for posteam and defteam
plays_games_subset = plays_games_subset.dropna(subset=['posteam', 'defteam'])
```

```
In [30]: plays_games_subset.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 457821 entries, 1 to 483604
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   game_id                457821 non-null object
1   play_id                457821 non-null float64
2   home_team              457821 non-null object
3   away_team              457821 non-null object
4   season_type            457821 non-null object
5   season                 457821 non-null int32
6   posteam                457821 non-null object
7   defteam                457821 non-null object
8   ydstogo                457821 non-null float64
9   down                   406918 non-null float64
10  play_type              449260 non-null object
11  score_differential      457791 non-null float64
12  epa                    455037 non-null float64
13  wp                     455041 non-null float64
14  wpa                     450674 non-null float64
15  penalty                448780 non-null float64
16  penalty_type           457821 non-null object
17  penalty_yards          457821 non-null float64
18  penalty_team           457821 non-null object
19  yards_gained           448780 non-null float64
20  replay_or_challenge     457821 non-null float64
21  replay_or_challenge_result 457821 non-null object
22  result                 457821 non-null int32
23  referee                457821 non-null object
dtypes: float64(11), int32(2), object(11)
memory usage: 83.8+ MB

```

We can now move forward with dropping remaining null values in the following variables, as NAs indicate missing data that could affect our analysis:

```

In [31]: # rows where play_type is NA
plays_games_subset = plays_games_subset.dropna(subset=['play_type'])

In [32]: plays_games_subset.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 449260 entries, 1 to 483603
Data columns (total 24 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   game_id               449260 non-null object
 1   play_id              449260 non-null float64
 2   home_team            449260 non-null object
 3   away_team            449260 non-null object
 4   season_type          449260 non-null object
 5   season               449260 non-null int32
 6   posteam              449260 non-null object
 7   defteam              449260 non-null object
 8   ydstogo              449260 non-null float64
 9   down                 406826 non-null float64
10  play_type            449260 non-null object
11  score_differential    449260 non-null float64
12  epa                  449258 non-null float64
13  wp                   449260 non-null float64
14  wpa                  444893 non-null float64
15  penalty              448779 non-null float64
16  penalty_type         449260 non-null object
17  penalty_yards        449260 non-null float64
18  penalty_team         449260 non-null object
19  yards_gained         448779 non-null float64
20  replay_or_challenge   449260 non-null float64
21  replay_or_challenge_result 449260 non-null object
22  result               449260 non-null int32
23  referee              449260 non-null object
dtypes: float64(11), int32(2), object(11)
memory usage: 82.3+ MB

```

```

In [33]: # NAs in yards_gained
plays_games_subset = plays_games_subset.dropna(subset=['yards_gained'])

```

```

In [34]: # NAs in epa
plays_games_subset = plays_games_subset.dropna(subset=['epa'])

```

```

In [35]: # NAs in wpa
plays_games_subset = plays_games_subset.dropna(subset=['wpa'])

```

```

In [36]: plays_games_subset.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 444411 entries, 1 to 483600
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   game_id                444411 non-null object
1   play_id                444411 non-null float64
2   home_team              444411 non-null object
3   away_team              444411 non-null object
4   season_type            444411 non-null object
5   season                 444411 non-null int32
6   posteam                444411 non-null object
7   defteam                444411 non-null object
8   ydstogo                444411 non-null float64
9   down                   402067 non-null float64
10  play_type              444411 non-null object
11  score_differential      444411 non-null float64
12  epa                    444411 non-null float64
13  wp                     444411 non-null float64
14  wpa                    444411 non-null float64
15  penalty                444411 non-null float64
16  penalty_type           444411 non-null object
17  penalty_yards          444411 non-null float64
18  penalty_team           444411 non-null object
19  yards_gained           444411 non-null float64
20  replay_or_challenge    444411 non-null float64
21  replay_or_challenge_result 444411 non-null object
22  result                 444411 non-null int32
23  referee                444411 non-null object
dtypes: float64(11), int32(2), object(11)
memory usage: 81.4+ MB

```

Correcting Entries

Upon further inspection, there are several entries that need to be corrected. First, we will look at the value counts of the 'referee' variable.

```
In [37]: plays_games_subset['referee'].value_counts()
```

```
Out[37]: referee
Clete Blakeman    26974
Bill Vinovich     26775
Carl Cheffers     26640
John Hussey       26446
Craig Wrolstad    26303
Brad Allen        25771
Ron Torbert       21983
Jerome Boger      20392
Shawn Hochuli     18593
Shawn Smith       18553
Alex Kemp         18520
Tony Corrente     18322
Clay Martin       17895
Adrian Hill       15850
Scott Novak       15385
Land Clark        12877
Walt Anderson     12841
Brad Rogers       12209
Walt Coleman      10193
Peter Morelli     10057
John Parry        9892
Ed Hochuli        8258
Gene Steratore    7875
Tra Blake         7725
Jeff Triplette    7504
Terry McAulay     7239
Alan Eck          5176
Ronald Torbert    5070
Bradley Rogers    2458
John Perry        310
Adrian Hall       164
Tony Steratore    161
Name: count, dtype: int64
```

As you can see, there are several instances where the head referee's name was entered incorrectly. So, we will map the correct names onto the current entries:

```
In [38]: # Fixing referee names
ref_name_mapping = {
    'Adrian Hall': 'Adrian Hill',
    'Bradley Rogers': 'Brad Rogers',
    'John Perry': 'John Parry',
    'Ronald Torbert': 'Ron Torbert',
}

plays_games_subset['referee'] = plays_games_subset['referee'].replace(ref_name_mapping)
```

Furthermore, Tony Steratore was never a head referee, but a back judge. We will also remove entries that include his name:

```
In [39]: # Keep only rows where referee is not Tony Steratore
plays_games_subset = plays_games_subset[plays_games_subset['referee'] != 'Tony Steratore']
```

```
In [40]: plays_games_subset['referee'].value_counts()
```

```
Out[40]: referee
Ron Torbert      27053
Clete Blakeman   26974
Bill Vinovich    26775
Carl Cheffers    26640
John Hussey      26446
Craig Wrolstad   26303
Brad Allen       25771
Jerome Boger     20392
Shawn Hochuli    18593
Shawn Smith      18553
Alex Kemp        18520
Tony Corrente    18322
Clay Martin      17895
Adrian Hill      16014
Scott Novak      15385
Brad Rogers      14667
Land Clark       12877
Walt Anderson    12841
John Parry       10202
Walt Coleman     10193
Peter Morelli    10057
Ed Hochuli       8258
Gene Steratore   7875
Tra Blake        7725
Jeff Triplette   7504
Terry McAulay    7239
Alan Eck         5176
Name: count, dtype: int64
```

2.5 Feature Engineering

We created two new variables to aid in our analysis, the first being the 'era' variable. This sorts teams by season into either 'pre-betting' (2015-2019) or 'post-betting' (2020-2024). We used 2019-2020 as the inflection point because, although states adopted legal sports betting on a staggered timeline, 2020 represents the point at which sports betting became nationally integrated into the NFL environment. From 2015–2019, the league operated in a largely betting-free context: PASPA remained in place until late 2018, sportsbook presence was minimal, and betting content was largely absent from broadcasts. Beginning in 2020, however, sports betting shifted from isolated early adoption to a mainstream national trend. Major operators expanded rapidly, betting odds and analysis became standard in NFL media coverage, and the league formalized partnerships with sportsbooks. Even teams located in non-legal states competed within this new national media and market environment. Grouping 2015–2019 as the pre-betting era and 2020–2024 as the post-betting era therefore captures the structural shift in the league's overall betting landscape and aligns with when betting began influencing fan behavior, broadcast narratives, and league operations at scale.

```
In [41]: # Defining era variable
plays_games_subset['era'] = np.where(plays_games_subset['season'] <= 2019,
                                     'pre_betting', 'post_betting')
```

Additionally, because there are many different possible penalty type entries, we created a new variable, 'penalty_category', to sort 'penalty_type' into the below categories:

- Offensive holding / Line of scrimmage
 - Offensive Holding, False Start, Offensive Pass Interference, Illegal Formation, Illegal Shift, Illegal Motion, Offensive Offside, Offensive Too Many Men on Field, Ineligible Downfield Pass, Illegal Double-Team Block, Offensive 12 On-field
- Defensive holding / Pass interference / Contact
 - Defensive Pass Interference, Defensive Holding, Illegal Contact, Roughing the Passer, Face Mask, Defensive Offside, Defensive Too Many Men on Field, Defensive Delay of Game, Defensive 12 On-field
- Personal / Roughness
 - Unnecessary Roughness, Unsportsmanlike Conduct, Taunting, Lowering the Head to Initiate Contact, Lowering the Head to Make forcible Contact, Horse Collar Tackle, Chop Block, Hip Drop Tackle, Leverage
- Kicking / Special teams
 - Offside on Free Kick, Running Into the Kicker, Roughing the Kicker, Kick Catch Interference, Player Out of Bounds on Kick, Player Out of Bounds on Punt, Illegal Touch Kick, Ineligible Downfield Kick, Illegal Kick/Kicking Loose Ball, Kickoff Short of Landing Zone, Illegal Wedge, Short Free Kick
- Procedural / Clock / Formation
 - Delay of Game, Neutral Zone Infraction, Encroachment, Illegal Substitution, Illegal Shift, Illegal Forward Pass, Intentional Grounding, Invalid Fair Catch Signal
- Illegal blocks / Contact above the waist
 - Illegal Block Above the Waist, Low Block, Clipping, Illegal Blindside Block, Illegal Crackback
- Other (rare)
 - Leaping, Fair Catch Interference, Illegal Bat, Tripping, Disqualification, Illegal Touch Pass

```
In [42]: # Defining mapping for penalty_category
penalty_mapping = {
    'Offensive Holding': 'Offensive/HOL', 'False Start': 'Offensive/HOL',
```

```

'Offensive Pass Interference':'Offensive/HOL', 'Illegal Formation':'Offensive/H
'Illegal Shift':'Offensive/HOL', 'Illegal Motion':'Offensive/HOL',
'Offensive Offside':'Offensive/HOL', 'Offensive Too Many Men on Field':'Offensi
'Ineligible Downfield Pass':'Offensive/HOL', 'Illegal Double-Team Block':'Offen
'Offensive 12 On-field':'Offensive/HOL',

'Defensive Pass Interference':'Defensive/PI', 'Defensive Holding':'Defensive/PI
'Illegal Contact':'Defensive/PI', 'Roughing the Passer':'Defensive/PI',
'Face Mask':'Defensive/PI', 'Defensive Offside':'Defensive/PI',
'Defensive Too Many Men on Field':'Defensive/PI', 'Defensive Delay of Game':'De
'Defensive 12 On-field':'Defensive/PI',

'Unnecessary Roughness':'Personal/Roughness', 'Unsportsmanlike Conduct':'Person
'Taunting':'Personal/Roughness', 'Lowering the Head to Initiate Contact':'Perso
'Lowering the Head to Make Forcible Contact':'Personal/Roughness', 'Horse Colla
'Chop Block':'Personal/Roughness', 'Hip Drop Tackle':'Personal/Roughness', 'Lev

'Offside on Free Kick':'Kicking/ST', 'Running Into the Kicker':'Kicking/ST',
'Roughing the Kicker':'Kicking/ST', 'Kick Catch Interference':'Kicking/ST',
'Player Out of Bounds on Kick':'Kicking/ST', 'Player Out of Bounds on Punt':'Ki
'Illegal Touch Kick':'Kicking/ST', 'Ineligible Downfield Kick':'Kicking/ST',
'Illegal Kick/Kicking Loose Ball':'Kicking/ST', 'Kickoff Short of Landing Zone'
'Illegal Wedge':'Kicking/ST', 'Short Free Kick':'Kicking/ST',

'Delay of Game':'Procedural', 'Neutral Zone Infraction':'Procedural', 'Encroach
'Illegal Substitution':'Procedural', 'Illegal Forward Pass':'Procedural', 'Inte
'Invalid Fair Catch Signal':'Procedural',

'Illegal Block Above the Waist':'Illegal Block', 'Low Block':'Illegal Block',
'Clipping':'Illegal Block', 'Illegal Blindside Block':'Illegal Block', 'Illegal

'Leaping':'Other', 'Fair Catch Interference':'Other', 'Illegal Bat':'Other',
'Tripping':'Other', 'Disqualification':'Other', 'Illegal Touch Pass':'Other'
}

# Applying mapping to create penalty_category
plays_games_subset['penalty_category'] = plays_games_subset['penalty_type'].map(pen

```

Now, our final dataset is ready for use in our analysis:

```
In [43]: plays_games_subset.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 444250 entries, 1 to 483600
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   game_id                444250 non-null object
1   play_id                444250 non-null float64
2   home_team              444250 non-null object
3   away_team              444250 non-null object
4   season_type            444250 non-null object
5   season                 444250 non-null int32
6   posteam                444250 non-null object
7   defteam                444250 non-null object
8   ydstogo                444250 non-null float64
9   down                   401929 non-null float64
10  play_type              444250 non-null object
11  score_differential      444250 non-null float64
12  epa                    444250 non-null float64
13  wp                     444250 non-null float64
14  wpa                    444250 non-null float64
15  penalty                444250 non-null float64
16  penalty_type           444250 non-null object
17  penalty_yards          444250 non-null float64
18  penalty_team           444250 non-null object
19  yards_gained           444250 non-null float64
20  replay_or_challenge    444250 non-null float64
21  replay_or_challenge_result 444250 non-null object
22  result                 444250 non-null int32
23  referee                444250 non-null object
24  era                    444250 non-null object
25  penalty_category       444250 non-null object
dtypes: float64(11), int32(2), object(13)
memory usage: 88.1+ MB

```

In [44]: `plays_games_subset.head()`

Out[44]:

	game_id	play_id	home_team	away_team	season_type	season	posteam	deft
--	---------	---------	-----------	-----------	-------------	--------	---------	------

1	2015_01_BAL_DEN	36.00	DEN	BAL	REG	2015	BAL	
2	2015_01_BAL_DEN	51.00	DEN	BAL	REG	2015	BAL	
3	2015_01_BAL_DEN	75.00	DEN	BAL	REG	2015	BAL	
4	2015_01_BAL_DEN	96.00	DEN	BAL	REG	2015	BAL	
5	2015_01_BAL_DEN	120.00	DEN	BAL	REG	2015	BAL	

5 rows × 26 columns



```
In [45]: ## un-comment to save to csv files
# plays_games_subset.to_csv("plays_games_subset.csv", index=False)
# teams.to_csv("teams.csv", index=False)
```

```
In [46]: # un-comment to open csv files
plays_games_subset = pd.read_csv("plays_games_subset.csv")
teams = pd.read_csv("teams.csv")
```

3. Exploratory Analysis & Findings

3.1 Analysis for Question 1: How have overall NFL penalty trends (rates, types, and yardage) changed between the pre-betting and post-betting era?

To begin our analysis, our group examined overall penalty trends for each season to understand how officiating patterns shifted between the pre-betting and post-betting eras.

```
In [47]: # Penalty rates per season

# Only keep plays with penalties
penalties = plays_games_subset[plays_games_subset['penalty'] == 1].copy()

# Penalty counts by season
penalty_by_season = penalties.groupby('season').size().reset_index(name = 'penalty_

# Total plays per season (counting entries)
total_plays_season = plays_games_subset.groupby('season').size().reset_index(name =

# Merge to compute penalty rates per 100 plays
penalty_rate_season = penalty_by_season.merge(
    total_plays_season,
    left_on = ['season'],
    right_on = ['season'],
    how = 'left'
)

penalty_rate_season['penalties_per_100'] = penalty_rate_season['penalty_count'] / p

# Display top penalty rates for inspection
penalty_rate_season.sort_values('season').head(10)
```

Out[47]:

	season	penalty_count	total_plays	penalties_per_100
0	2015	3660	44294	8.26
1	2016	3541	43928	8.06
2	2017	3499	43301	8.08
3	2018	3564	43234	8.24
4	2019	3565	43473	8.20
5	2020	2984	43861	6.80
6	2021	3326	45719	7.27
7	2022	3151	45362	6.95
8	2023	3226	45668	7.06
9	2024	3642	45410	8.02

In [48]:

```

# Penalty rates per team and season

# Penalty counts by team and season
penalty_by_team = penalties.groupby(['penalty_team', 'season']).size().reset_index()

# Total plays by team per season (combining counts posteam and defteam to get total)
total_plays = plays_games_subset.melt(
    id_vars = 'season',
    value_vars = ['home_team', 'away_team'],
    value_name = 'team'
).groupby(['team', 'season']).size().reset_index(name = 'total_plays')

# Merge to compute penalty rates per 100 plays
penalty_rate = penalty_by_team.merge(
    total_plays,
    left_on = ['penalty_team', 'season'],
    right_on = ['team', 'season'],
    how = 'left'
)

penalty_rate['penalties_per_100'] = penalty_rate['penalty_count'] / penalty_rate['total_plays'] * 100

# Display top penalty rates for inspection
penalty_rate.sort_values('penalties_per_100', ascending=False).head(10)

```

Out[48]:

	penalty_team	season	penalty_count	team	total_plays	penalties_per_100
272	SEA	2017	146	SEA	2670	5.47
30	BUF	2015	142	BUF	2629	5.40
181	LV	2016	154	LV	2857	5.39
290	TB	2015	142	TB	2661	5.34
192	MIA	2017	136	MIA	2573	5.29
249	NYJ	2024	137	NYJ	2674	5.12
144	JAX	2019	132	JAX	2589	5.10
184	LV	2019	128	LV	2511	5.10
180	LV	2015	139	LV	2729	5.09
190	MIA	2015	134	MIA	2679	5.00

```

In [49]: # Compute total penalties per team within each era
penalty_era = penalties.groupby(['penalty_team', 'era']).size().reset_index(name =

# Compute total plays per team within each era
total_plays_era = plays_games_subset.melt(
    id_vars = 'era',
    value_vars = ['home_team', 'away_team'],
    value_name = 'team'
).groupby(['team', 'era']).size().reset_index(name = 'total_plays')

# Merge to compute penalty rates per 100 plays
penalty_rates_era = penalty_era.merge(
    total_plays_era,
    left_on = ['penalty_team', 'era'],
    right_on = ['team', 'era'],
    how = 'left'
)

penalty_rates_era['penalties_per_100'] = penalty_rates_era['penalty_count'] / penal

# Merge with teams
penalty_rates_era_plot = penalty_rates_era.merge(
    teams[['team_abbr', 'team_color', 'team_logo_espn']],
    left_on = 'penalty_team',
    right_on = 'team_abbr',
    how = 'left'
)

penalty_rates_era_plot.head(10)

```

Out[49]:

	penalty_team	era	penalty_count	team	total_plays	penalties_per_100	team_at
0	ARI	post_betting	543	ARI	13794	3.94	/
1	ARI	pre_betting	533	ARI	13432	3.97	/
2	ATL	post_betting	425	ATL	13495	3.15	/
3	ATL	pre_betting	549	ATL	13898	3.95	/
4	BAL	post_betting	558	BAL	14599	3.82	E
5	BAL	pre_betting	577	BAL	13622	4.24	E
6	BUF	post_betting	569	BUF	15162	3.75	B
7	BUF	pre_betting	593	BUF	13241	4.48	B
8	CAR	post_betting	527	CAR	13356	3.95	C
9	CAR	pre_betting	485	CAR	13779	3.52	C

In [50]:

```

from matplotlib.offsetbox import OffsetImage, AnnotationBbox
import matplotlib.image as mpimg
from io import BytesIO
import urllib.request

# Determine the maximum penalty rate across both eras (for consistent y-axis)
max_penalty_rate = penalty_rates_era_plot['penalties_per_100'].max() * 1.1

# Function to plot penalty rate per team with colors/logos
def plot_penalty_rate(team_plot, era_name, y_max):
    # Sort by penalty rate
    team_plot = team_plot.sort_values('penalties_per_100', ascending = False)

    fig, ax = plt.subplots(figsize=(14,6))
    bars = ax.bar(
        team_plot['penalty_team'],
        team_plot['penalties_per_100'],
        color = team_plot['team_color'], # uses color codes from 'teams'
    )

    ax.set_title(f"Penalty Rate by Team ({era_name})", fontsize = 16)
    ax.set_ylabel("Penalty Rate (per 100 plays)", fontsize = 14)
    ax.set_xlabel("Team", fontsize = 14)
    plt.xticks(rotation = 45, ha = 'right')

    # finding average penalty rate
    avg_rate = team_plot["penalties_per_100"].mean()
    num_teams = len(team_plot)

    # Adding average line
    ax.axhline(avg_rate, color = "red", linestyle = "--", linewidth = 1.5)
    ax.text(num_teams, avg_rate, f"Average: {avg_rate:.3f}",
           color = "red", fontsize = 12, va = "bottom", ha = "right")

```

```

plt.grid(False) #remove grid after adding average line

# Add Logos above bars
for bar, logo_url in zip(bars, team_plot['team_logo_espn']):
    # skip if no logo url
    if logo_url:
        try:
            # get image from url (in 'team_logo_espn')
            with urllib.request.urlopen(logo_url) as url:
                img = mpimg.imread(BytesIO(url.read()))

            # desired logo height
            desired_height = 20 # found with trial and error
            img_height, img_width = img.shape[0], img.shape[1] # original image
            # Compute zoom factor so the larger dimension matches desired height
            zoom_factor = desired_height / max(img_height, img_width)
            # Create an image object with scaling
            imagebox = OffsetImage(img, zoom = zoom_factor)

            # x-position: centered on bar
            bar_x = bar.get_x() + bar.get_width()/2
            # y-position: slightly above top of bar
            bar_y = bar.get_height() + 0.001

            # Creating annotation box for placing the image on the plot
            ab = AnnotationBbox(
                imagebox,
                (bar_x, bar_y),
                frameon = False, # no bounding box
                box_alignment = (0.5, 0) # center horizontally, attach at bottom
            )
            # Adding logo to plot
            ax.add_artist(ab)
        except:
            pass

# Ensure enough vertical space
ax.set_ylim(0, y_max)
plt.xticks(rotation = 45)
plt.tight_layout()
plt.show()

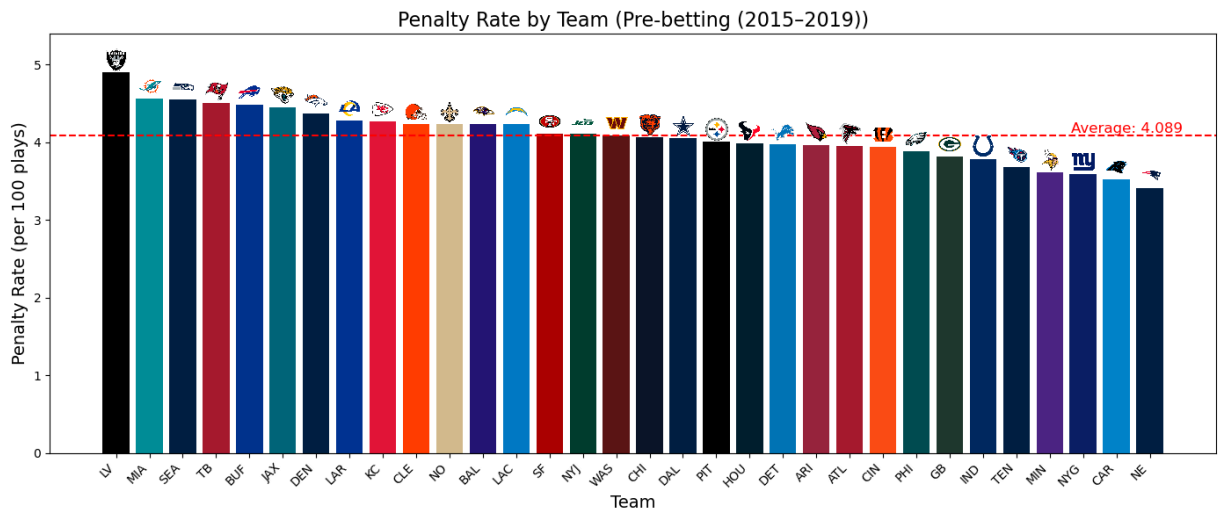
```

Figure 3.1.1A

```

In [51]: # Pre-betting Era
pre_team_plot = penalty_rates_era_plot[penalty_rates_era_plot['era'] == 'pre_bettin
plot_penalty_rate(pre_team_plot, "Pre-betting (2015-2019)", max_penalty_rate)

```



```
In [52]: # For precise numbers
pre_team_plot.head(32)
```

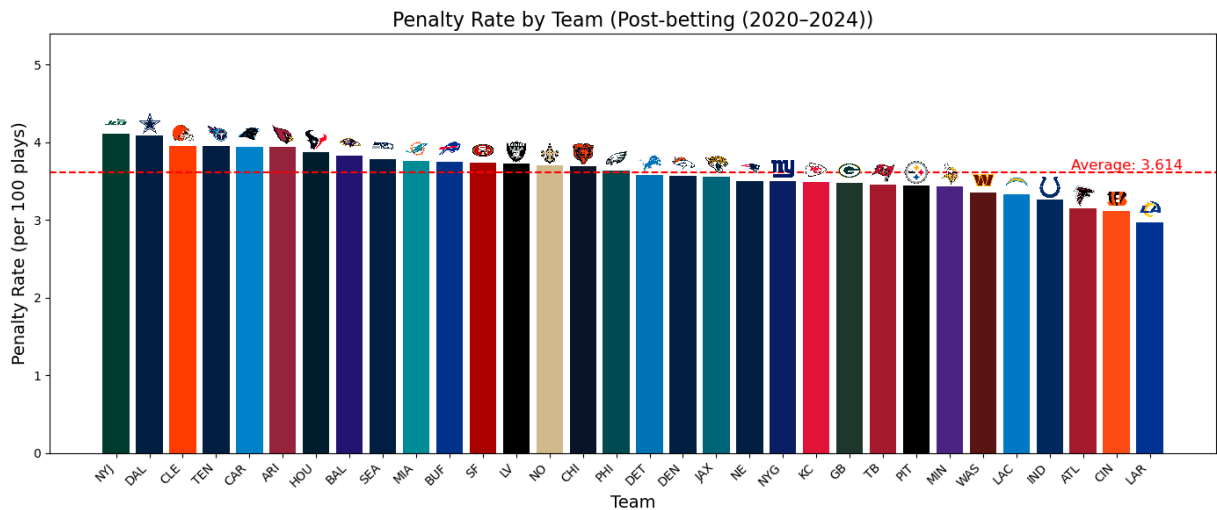
Out[52]:

	penalty_team	era	penalty_count	team	total_plays	penalties_per_100	team_al
1	ARI	pre_betting	533	ARI	13432	3.97	.
3	ATL	pre_betting	549	ATL	13898	3.95	.
5	BAL	pre_betting	577	BAL	13622	4.24	E
7	BUF	pre_betting	593	BUF	13241	4.48	E
9	CAR	pre_betting	485	CAR	13779	3.52	C
11	CHI	pre_betting	527	CHI	12962	4.07	(
13	CIN	pre_betting	523	CIN	13270	3.94	(
15	CLE	pre_betting	558	CLE	13170	4.24	(
17	DAL	pre_betting	542	DAL	13367	4.05	D
19	DEN	pre_betting	598	DEN	13684	4.37	D
21	DET	pre_betting	520	DET	13074	3.98	D
23	GB	pre_betting	539	GB	14138	3.81	
25	HOU	pre_betting	568	HOU	14250	3.99	H
27	IND	pre_betting	511	IND	13494	3.79	I
29	JAX	pre_betting	611	JAX	13729	4.45	.
31	KC	pre_betting	623	KC	14588	4.27	
33	LAC	pre_betting	558	LAC	13193	4.23	L
35	LAR	pre_betting	591	LAR	13807	4.28	L
37	LV	pre_betting	644	LV	13131	4.90	
39	MIA	pre_betting	600	MIA	13142	4.57	M
41	MIN	pre_betting	490	MIN	13572	3.61	M
43	NE	pre_betting	522	NE	15326	3.41	
45	NO	pre_betting	590	NO	13926	4.24	
47	NYG	pre_betting	487	NYG	13559	3.59	N
49	NYJ	pre_betting	537	NYJ	13075	4.11	I
51	PHI	pre_betting	559	PHI	14380	3.89	I
53	PIT	pre_betting	560	PIT	13959	4.01	
55	SEA	pre_betting	641	SEA	14100	4.55	S
57	SF	pre_betting	561	SF	13646	4.11	
59	TB	pre_betting	596	TB	13215	4.51	

	penalty_team	era	penalty_count	team	total_plays	penalties_per_100	team_al
61	TEN	pre_betting	502	TEN	13650	3.68	T
63	WAS	pre_betting	534	WAS	13081	4.08	W

Figure 3.1.1B

```
In [53]: # Post-betting Era
post_team_plot = penalty_rates_era_plot[penalty_rates_era_plot['era'] == 'post_bett
plot_penalty_rate(post_team_plot, "Post-betting (2020-2024)", max_penalty_rate)
print(plot_penalty_rate)
```



```
<function plot_penalty_rate at 0xffa2efa6b7e0>
```

```
In [54]: # For precise numbers
post_team_plot.head(32)
```

Out[54]:

	penalty_team	era	penalty_count	team	total_plays	penalties_per_100	team_s
0	ARI	post_betting	543	ARI	13794	3.94	
2	ATL	post_betting	425	ATL	13495	3.15	
4	BAL	post_betting	558	BAL	14599	3.82	
6	BUF	post_betting	569	BUF	15162	3.75	
8	CAR	post_betting	527	CAR	13356	3.95	
10	CHI	post_betting	492	CHI	13345	3.69	
12	CIN	post_betting	452	CIN	14507	3.12	
14	CLE	post_betting	558	CLE	14101	3.96	
16	DAL	post_betting	599	DAL	14659	4.09	
18	DEN	post_betting	486	DEN	13622	3.57	
20	DET	post_betting	508	DET	14211	3.57	
22	GB	post_betting	486	GB	13965	3.48	
24	HOU	post_betting	545	HOU	14070	3.87	t
26	IND	post_betting	445	IND	13642	3.26	
28	JAX	post_betting	491	JAX	13816	3.55	
30	KC	post_betting	561	KC	16071	3.49	
32	LAC	post_betting	465	LAC	13974	3.33	
34	LAR	post_betting	439	LAR	14764	2.97	
36	LV	post_betting	511	LV	13708	3.73	
38	MIA	post_betting	516	MIA	13720	3.76	
40	MIN	post_betting	485	MIN	14135	3.43	
42	NE	post_betting	458	NE	13072	3.50	
44	NO	post_betting	510	NO	13788	3.70	
46	NYG	post_betting	482	NYG	13763	3.50	
48	NYJ	post_betting	555	NYJ	13518	4.11	
50	PHI	post_betting	549	PHI	15111	3.63	
52	PIT	post_betting	490	PIT	14209	3.45	
54	SEA	post_betting	530	SEA	13995	3.79	
56	SF	post_betting	544	SF	14542	3.74	
58	TB	post_betting	530	TB	15341	3.45	

	penalty_team	era	penalty_count	team	total_plays	penalties_per_100	team_e
60	TEN	post_betting	544	TEN	13774	3.95	
62	WAS	post_betting	476	WAS	14211	3.35	

The first aspect of the penalty trends we wanted to explore was penalty rates. We decided that the best way to compare penalty rates was to look at the penalty rates for each team per 100 plays. This gives us an easily comparable metric for each team split between the pre- and post-betting era. Figure 3.1.1A specifically targets the penalty rate for each team per 100 plays in the pre-betting era, with a dashed line showcasing the mean penalty rate. In the pre-betting era, the NFL saw an average of around 4.1 penalties per 100 plays. This is followed by a shift in the post-betting era, as shown by Figure 3.1.1B, with penalties per 100 plays shifting to an average of 3.6 across the league. When comparing specific teams in the post-betting era to the pre-betting average penalty rate, only one team broke the 4.1 average, the New York Jets. Additionally, only one other team, the Dallas Cowboys, even had a penalty rate per 100 plays higher than 4. That means the rest of the 30 NFL teams had penalty rates per 100 plays below 4, a figure lower than the average penalty rate a few years prior. Overall, Figures 3.1.1A and 3.1.1B show that there is a clear difference in the average penalty rates between the pre- and post-adoption of legalized sports betting in the U.S.

Figure 3.1.2A

```
In [55]: # Split the dataset into pre-betting (2015-2019) and post-betting (2020-2024) eras
pre = plays_games_subset[plays_games_subset["era"] == "pre_betting"]
post = plays_games_subset[plays_games_subset["era"] == "post_betting"]

# Count the number of games in each era
pre_games = pre["game_id"].nunique()
post_games = post["game_id"].nunique()

# Filter to only plays with penalties
pre_pen = pre[pre["penalty"] == 1]
post_pen = post[post["penalty"] == 1]

# Count how many times each penalty type appears in each era
pre_counts = pre_pen["penalty_type"].value_counts()
post_counts = post_pen["penalty_type"].value_counts()

# Convert counts into penalties per game
pre_avg = pre_counts / pre_games
post_avg = post_counts / post_games

# Get the top 5 most common penalties in each era
pre_top5 = pre_avg.sort_values(ascending=False).head(5)
post_top5 = post_avg.sort_values(ascending=False).head(5)

# Plot the pre-betting era top 5 penalties
plt.figure(figsize=(8, 7))
```

```
plt.bar(pre_top5.index, pre_top5.values, color="Blue")

# Add numbers above each bar
for i, v in enumerate(pre_top5.values):
    plt.text(i, v + 0.01, round(v, 2), ha='center')

# Graph formatting
plt.title("Top 5 Penalties Per Game (Pre-Betting Era: 2015-2019)")
plt.xlabel("Penalty Type")
plt.ylabel("Avg Penalties Per Game")
plt.xticks(rotation=45, ha="right")
plt.show()
```

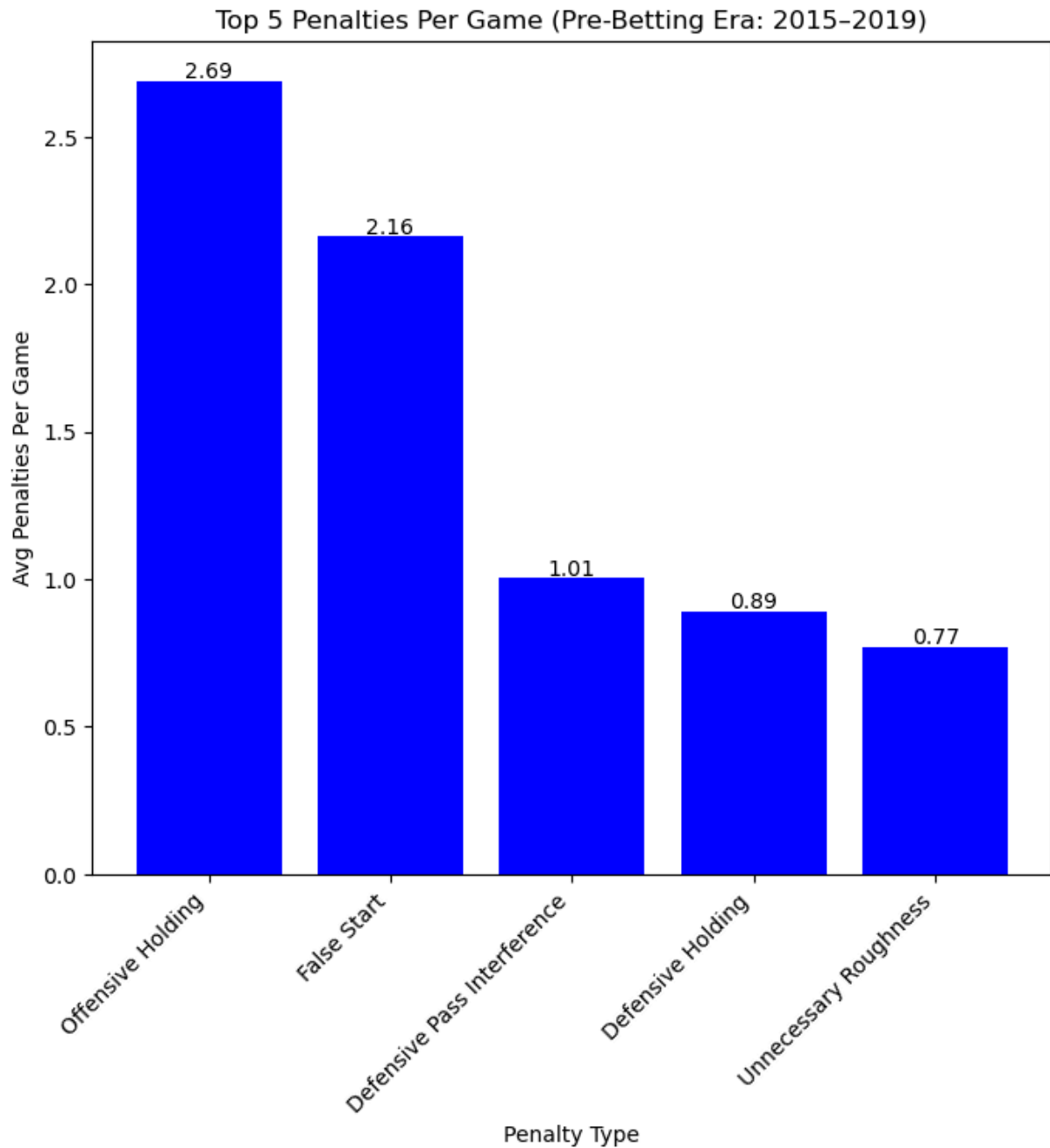
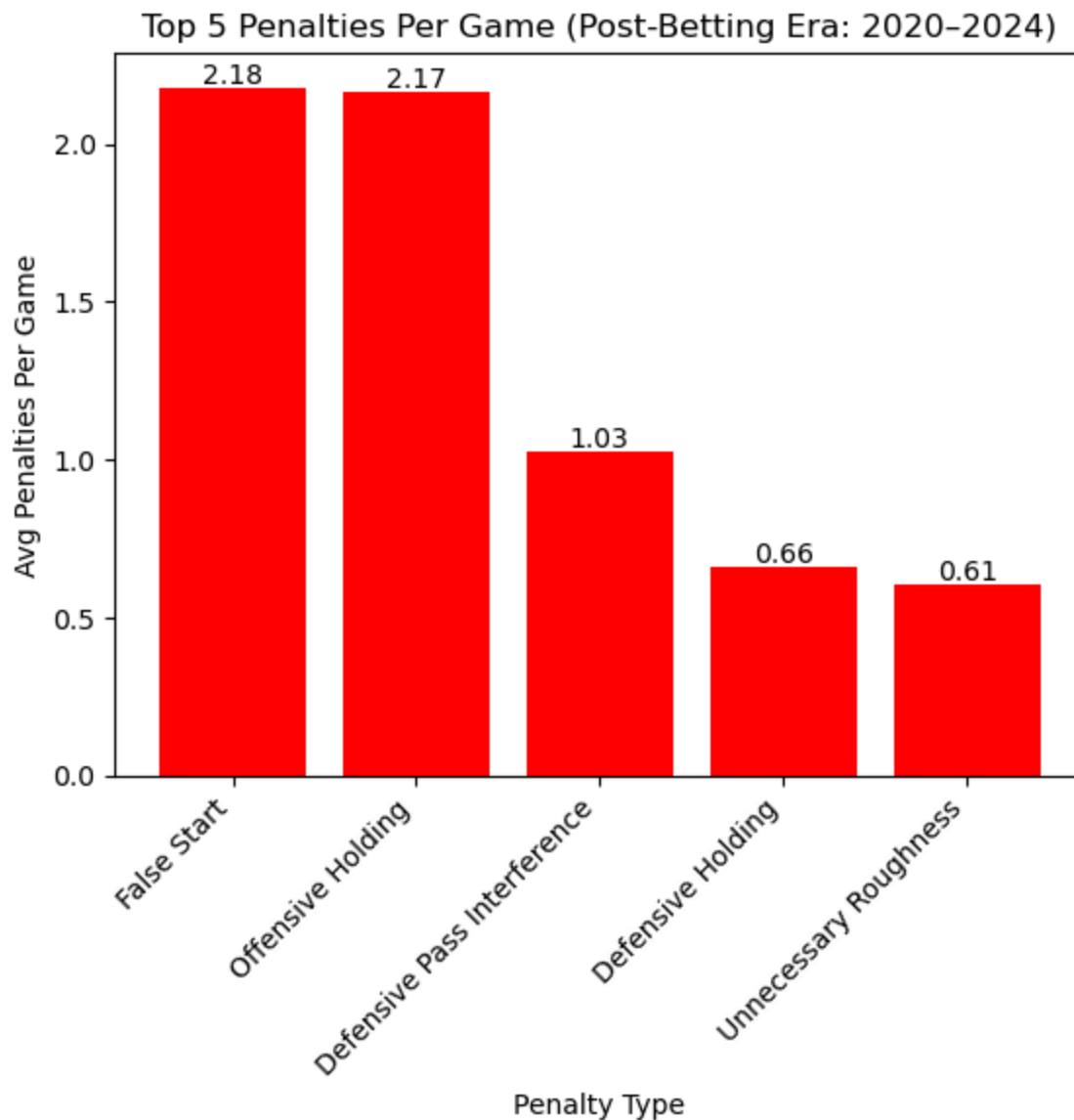


Figure 3.1.2B

```
In [56]: # Plot the post-betting era top 5 penalties
plt.bar(post_top5.index, post_top5.values, color="red")

# Add numbers above each bar
for i, v in enumerate(post_top5.values):
    plt.text(i, v + 0.01, round(v, 2), ha='center')

# Graph formatting
plt.title("Top 5 Penalties Per Game (Post-Betting Era: 2020-2024)")
plt.xlabel("Penalty Type")
plt.ylabel("Avg Penalties Per Game")
plt.xticks(rotation=45, ha="right")
plt.figure(figsize=(8, 7))
plt.show()
```



<Figure size 800x700 with 0 Axes>

To investigate this difference further, we chose to look at the top 5 most called penalties split between eras in Figure 3.1.2A and 3.1.2B. This was used to see if the number of most commonly called penalties was related to the large difference seen between 3.1.1A and

3.1.1B. We broke down each penalty by the average number of times it was called per game to be consistent with the graphs from Q3.1. In both eras, the top 5 most called penalties remained constant by type and order. The most called penalty in both eras, offensive holding, had the largest change in average calls per game. From the pre-betting era to the post-betting era, the average number of calls per game for offensive holding decreased from 2.69 to 2.17. This isn't a very large change, and the other four penalties showed even less of a difference. This indicates that the decline in penalties per game from the pre- to the post-betting era wasn't driven by a restructuring in the most commonly called penalties.

Figure 3.1.3

```
In [57]: # Keep only plays with penalties
penalties = plays_games_subset[plays_games_subset["penalty"] == 1]

# Get total penalty yards per season
yards_per_season = penalties.groupby("season")["penalty_yards"].sum()

# Get number of games
games = plays_games_subset.groupby("season")["game_id"].nunique()

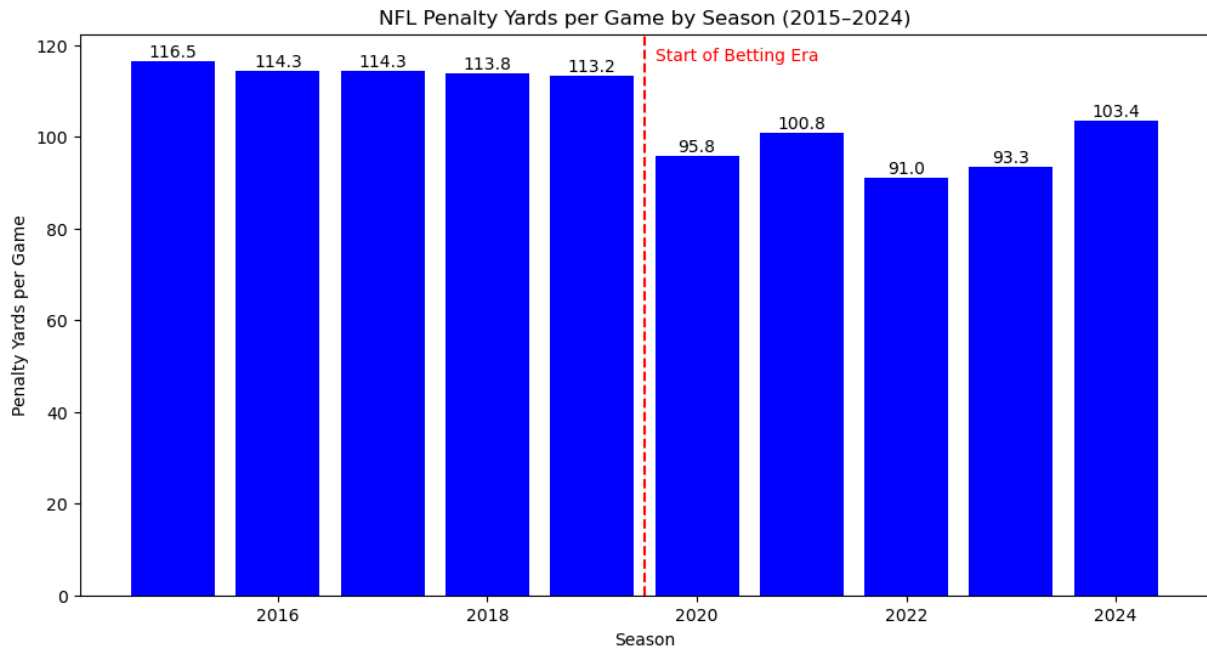
# Use above to get penalty yards per game
avg_yards_per_game = yards_per_season / games

# Creating and formatting chart
plt.figure(figsize=(12, 6))
bars = plt.bar(avg_yards_per_game.index, avg_yards_per_game.values,
               color="Blue")

# Values at the top of each bar
for i, value in enumerate(avg_yards_per_game.values):
    plt.text(avg_yards_per_game.index[i],
             value + 1,
             round(value, 1),
             ha="center")

# Add vertical era defining line
plt.axvline(x=2019.5, color='red', linestyle='--')
plt.text(2019.6, avg_yards_per_game.max(), "Start of Betting Era", color='red')

# Graph formatting
plt.title("NFL Penalty Yards per Game by Season (2015-2024)")
plt.xlabel("Season")
plt.ylabel("Penalty Yards per Game")
plt.show()
```



Lastly, we decided to investigate the average number of penalty yards enforced per game. After seeing that the most common penalties remained similar, we wanted to investigate if the fewer penalties in the post-betting era from Q3.1 reduced the average penalty yardage enforced per game. This is to see if the officials compensated for the fewer penalty calls by throwing penalties that were worth more yards. As seen in Figure 3.1.3, the penalty yardage tells a similar story to penalty count. The pre-betting era saw penalty yards per game between 113-116 yards per game, while the post-betting era saw only 91-103 yards per game. This coincides with the story that the decline in penalty frequency was accompanied by a decline in penalty yardage enforced, not just a shift toward larger-yardage penalties.

Overall, penalty trends showed both a decline in frequency and yardage in the seasons immediately following the start of the betting era, with the distribution of commonly called penalties remaining the same. These findings will feed into our next section, where we see if this league-wide decline is reflected uniformly across officiating crews, or whether certain crews changed their penalty-calling tendencies.

3.2 Analysis for Question 2: Do certain officiating crews differ in their frequency of calling penalties, and have these tendencies changed between the pre-betting and post-betting era?

To investigate whether officiating crews differ in their propensity to call penalties, and whether these tendencies have shifted between the pre-betting (2015–2019) and post-betting (2020–2024) eras, we began by examining league-wide penalty rates at the referee level. Penalty rate was measured as penalties per 100 plays, which adjusts for season length and the number of plays officiated by each crew.

Overall Penalty Rate Variation Across Referees

We begin with a league-wide view of penalty-calling behavior. Figure 3.2.1 displays each referee's aggregate penalty rate across all seasons in our dataset (2015–2024). The distribution reveals substantial variation across crews, with some referees averaging below 7 penalties per 100 plays, while others almost 9 (average = 7.755). This spread demonstrates that officiating crews do not enforce rules uniformly. Certain referees consistently call more penalties regardless of team, game situation, or season. The aggregate penalty-rate chart also reveals that not all referees worked the same number of seasons during the 2015–2024 window, which is to be expected. Some officials have only three or four seasons of data, while others appear across the entire decade. This uneven coverage naturally introduces variation, since officials with fewer games tend to have more volatile penalty rates. However, even when we restrict attention to the referees who were active throughout the full period, we still observe meaningful differences in penalty-calling tendencies. For instance, Bill Vinovich averaged 6.62 penalty calls per 100 plays across these ten seasons, while Carl Cheffers averaged 8.03 within the same time frame. This indicates that the variation in penalty rates is not solely a product of limited sample sizes.

```
In [58]: ## Penalty Rate by Referee

# Total plays per crew
crew_total = plays_games_subset.groupby('referee').size().reset_index(name = 'total_plays_per_crew')

# Total penalties per crew
crew_penalties = penalties.groupby('referee').size().reset_index(name = 'total_penalties_per_crew')

# Penalty rate per crew
crew_penalty_rate = crew_total.merge(crew_penalties, on = 'referee', how = 'left')

crew_penalty_rate['penalty_rate'] = crew_penalty_rate['total_penalties'] / crew_penalty_rate['total_plays_per_crew']

# Adding penalties_per_100 (to visualize more easily)
crew_penalty_rate['penalties_per_100'] = crew_penalty_rate['penalty_rate'] * 100

# Finding number of seasons each referee worked
seasons_active = plays_games_subset.groupby("referee")["season"].nunique().reset_index()
seasons_active.columns = ["referee", "seasons_active"]

# merge into crew_penalty_rate
crew_penalty_rate = crew_penalty_rate.merge(
    seasons_active,
    on = "referee",
    how = "left"
)
```

```
In [59]: crew_penalty_rate.sort_values('penalties_per_100')
```

Out[59]:

	referee	total_plays	total_penalties	penalty_rate	penalties_per_100	seasons_active
3	Bill Vinovich	26775	1773	0.07	6.62	10
1	Alan Eck	5176	354	0.07	6.84	2
19	Scott Novak	15385	1098	0.07	7.14	6
11	Gene Steratore	7875	566	0.07	7.19	3
14	John Hussey	26446	1949	0.07	7.37	10
21	Shawn Smith	18553	1375	0.07	7.41	7
7	Clay Martin	17895	1331	0.07	7.44	7
16	Land Clark	12877	970	0.08	7.53	5
4	Brad Allen	25771	1947	0.08	7.56	10
24	Tra Blake	7725	584	0.08	7.56	3
9	Craig Wrolstad	26303	1990	0.08	7.57	10
23	Tony Corrente	18322	1394	0.08	7.61	7
18	Ron Torbert	27053	2069	0.08	7.65	10
5	Brad Rogers	14667	1130	0.08	7.70	6
0	Adrian Hill	16014	1236	0.08	7.72	6
8	Clete Blakeman	26974	2112	0.08	7.83	10
20	Shawn Hochuli	18593	1475	0.08	7.93	7
13	Jerome Boger	20392	1622	0.08	7.95	8
26	Walt Coleman	10193	811	0.08	7.96	4
2	Alex Kemp	18520	1478	0.08	7.98	7

	referee	total_plays	total_penalties	penalty_rate	penalties_per_100	seasons_active
17	Peter Morelli	10057	806	0.08	8.01	4
6	Carl Cheffers	26640	2140	0.08	8.03	10
15	John Parry	10202	821	0.08	8.05	4
10	Ed Hochuli	8258	683	0.08	8.27	3
22	Terry McAulay	7239	626	0.09	8.65	3
12	Jeff Triplette	7504	664	0.09	8.85	3
25	Walt Anderson	12841	1154	0.09	8.99	5

```
In [60]: crew_penalty_rate_10 = (
          crew_penalty_rate[crew_penalty_rate['seasons_active'] == 10]
          .sort_values('penalties_per_100').reset_index(drop = True)
          )
          crew_penalty_rate_10
```

```
Out[60]:
```

	referee	total_plays	total_penalties	penalty_rate	penalties_per_100	seasons_active
0	Bill Vinovich	26775	1773	0.07	6.62	10
1	John Hussey	26446	1949	0.07	7.37	10
2	Brad Allen	25771	1947	0.08	7.56	10
3	Craig Wrolstad	26303	1990	0.08	7.57	10
4	Ron Torbert	27053	2069	0.08	7.65	10
5	Clete Blakeman	26974	2112	0.08	7.83	10
6	Carl Cheffers	26640	2140	0.08	8.03	10

Figure 3.2.1

```
In [61]: # penalty rate by crew (coloring bars by number of seasons active)
```

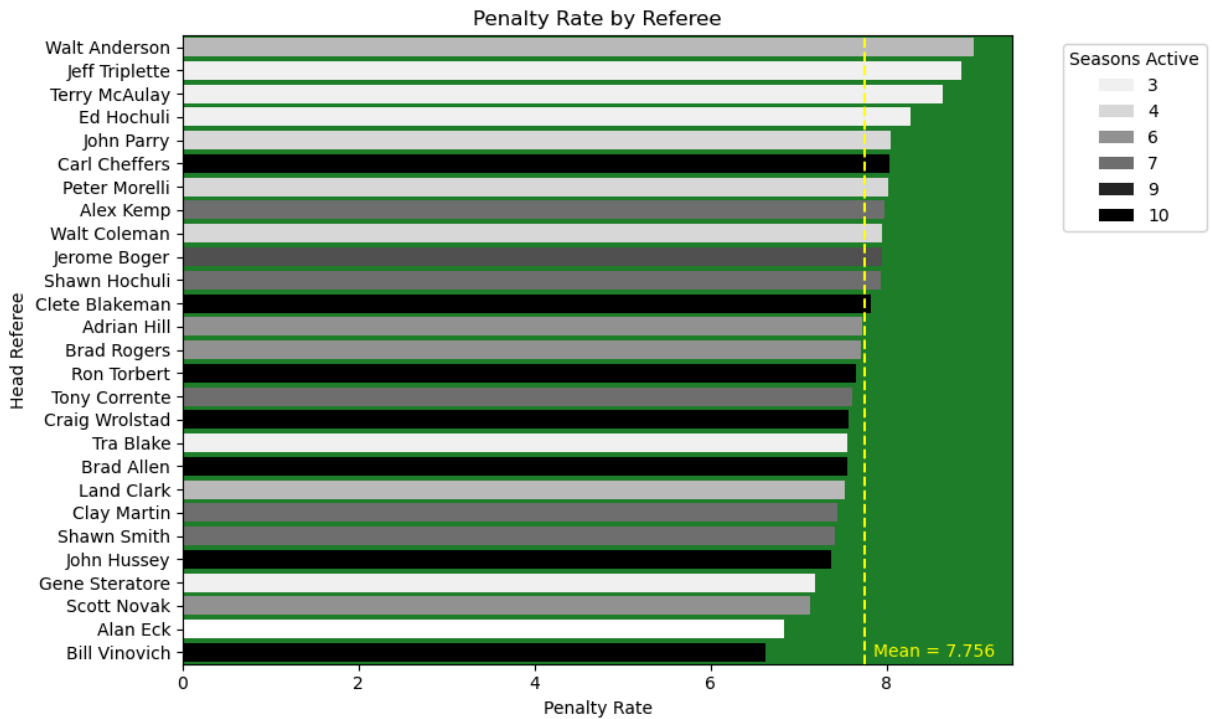
```
plt.figure(figsize = (10,6))

sns.barplot(
    data = crew_penalty_rate.sort_values('penalties_per_100', ascending = False),
    y = 'referee',
    x = 'penalties_per_100',
    hue = 'seasons_active',
    palette = 'Greys'
)

# Calculate overall mean penalty rate
mean_penalty = crew_penalty_rate['penalties_per_100'].mean()

# Add vertical line at the mean
num_crews = len(crew_penalty_rate)
ax = plt.gca()
ax.set_facecolor("#1e7f28") # turf color
ax.axvline(mean_penalty, color = 'yellow', linestyle = '--', linewidth = 1.5)
ax.text(
    mean_penalty + 0.1,
    num_crews - 1,
    f'Mean = {mean_penalty:.3f}',
    color = 'yellow',
    va = 'center',
    ha = 'left'
)

plt.title("Penalty Rate by Referee")
plt.xlabel("Penalty Rate")
plt.ylabel("Head Referee")
plt.legend(title = "Seasons Active", bbox_to_anchor = (1.05, 1), loc = 'upper left')
plt.tight_layout()
# plt.savefig("crew_penalty_rate_season.png", dpi=300, bbox_inches="tight")
plt.show()
```



Descriptive Statistics by Era

To provide context for the individual-level variation, we next summarize the league-wide penalty environment. Across all plays from 2015–2024:

```
In [62]: # Penalty rate by crew and era

# Compute penalty rate per referee per era using row counts
ref_era_stats = plays_games_subset.groupby(['referee', 'era']).agg(
    total_penalties=('penalty', 'sum'), # sum of penalty flags
    total_plays=('game_id', 'size')     # count rows
).reset_index()

# Calculate penalties per 100 plays
ref_era_stats['penalties_per_100'] = ref_era_stats['total_penalties'] / ref_era_stats['total_plays'] * 100

# Descriptive stats across referees per era
era_stats = ref_era_stats.groupby('era')['penalties_per_100'].agg(['mean', 'median', 'std'])
print(era_stats)
```

	era	mean	median	std
0	post_betting	7.19	7.26	0.40
1	pre_betting	8.20	8.11	0.62

As shown above, the mean penalty rate decreased from 8.20 in the pre-betting era (2015–2019) to 7.19 in the post-betting era (2020–2024). Similarly, the median penalty rate dropped from 8.11 to 7.26 across eras. The standard deviation also decreased from 0.62 to 0.40, indicating that referee penalty rates became more tightly clustered in the post-betting era. These numbers suggest two patterns. First, there is an overall decline in penalty frequency per referee after 2020. Second, the reduction in variation implies that referees' behaviors are more consistent with each other in the post-betting era. However, it is

important to note that while the differences between eras are measurable, the data do not imply causation. Other factors, such as rule changes, replay protocols, or league emphasis on game flow, may contribute to these shifts.

Crew-Team Interaction

While aggregate penalty rates reveal general tendencies, they may mask variation in how referees officiate specific teams. To examine this, we constructed a heatmap (Figure 3.2.2) showing penalties per 100 plays for every referee × team pair.

Figure 3.2.2

```
In [63]: # Constructing referee x team heatmap

# Building offensive/defensive play counts
offense_plays = plays_games_subset[['game_id', 'play_id', 'referee', 'posteam', 'er
defense_plays = plays_games_subset[['game_id', 'play_id', 'referee', 'defteam', 'er

# Combine
all_team_plays = pd.concat([offense_plays, defense_plays], ignore_index = True)

# heatmap of ref * team regardless of era

crew_team_plays = (
    all_team_plays
    .groupby(['referee', 'team'])
    .size()
    .reset_index(name = 'total_plays')
)

# Counting penalties against teams
crew_team_penalties = (
    plays_games_subset[plays_games_subset['penalty'] == 1]
    .groupby(['referee', 'penalty_team'])
    .size()
    .reset_index(name = 'penalty_count')
)

crew_team_penalties = crew_team_penalties.rename(columns = {'penalty_team': 'team'})

# Merge and calculate rates
crew_team_stats = crew_team_penalties.merge(
    crew_team_plays,
    on = ['referee', 'team'],
    how = 'left'
)

crew_team_stats['penalties_per_100'] = (crew_team_stats['penalty_count'] / crew_tea

# Only using refs who worked on all 32 teams
ref_counts = (
    crew_team_stats.groupby('referee')['team']
    .nunique()
```

```

    .reset_index(name = 'num_teams_seen')
)

valid_refs = ref_counts[ref_counts['num_teams_seen'] == 32]['referee'].tolist()

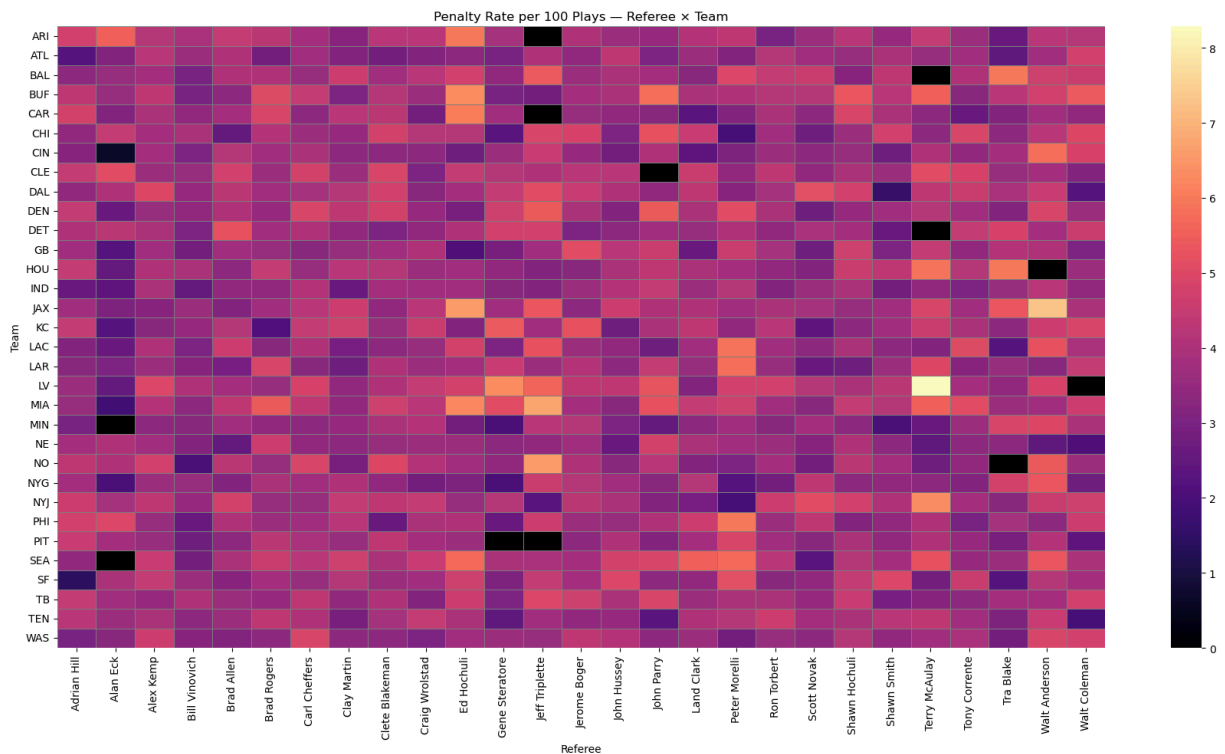
crew_team_stats_filtered = crew_team_stats[crew_team_stats['referee'].isin(valid_refs)]

# Pivot to heatmap format
heatmap_df = crew_team_stats.pivot(
    index='team',      # rows = NFL teams
    columns='referee',  # columns = referees
    values='penalties_per_100'
).fillna(0)

# Plot heatmap
plt.figure(figsize=(18, 10))
sns.heatmap(
    heatmap_df,
    cmap="magma",
    linewidths=0.5,
    linecolor='gray'
)

plt.title("Penalty Rate per 100 Plays – Referee × Team")
plt.xlabel("Referee")
plt.ylabel("Team")
plt.tight_layout()
# plt.savefig("ref_team_heatmap.png", dpi=300, bbox_inches="tight")
plt.show()

```



The heatmap shows that:

- Some referees call more penalties overall, as evidenced by lighter vertical bands (e.g., Jeff Triplette or Walt Anderson vs. Bill Vinovich or Alan Eck).
- Some teams draw more penalties regardless of referee, as indicated by horizontal bands (e.g., Las Vegas or Miami vs. Minnesota or New England).
- Certain referee–team combinations show unusually high or low penalty rates (e.g., Terry McAulay and Las Vegas, Brad Rogers and Kansas City).

These patterns indicate that some referees systematically call more penalties, and some teams consistently accumulate more penalties, but interactions between specific crews and teams also contribute to the variability. This granular view reinforces the need to analyze changes over time, since shifts in these tendencies may reveal behavior specific to the post-betting era.

Slope Plot: Pre-Betting vs. Post-Betting Changes by Referee

To directly assess how penalty-calling changed after 2019, we constructed a slope plot (Figure 3.3.3) connecting each referee's penalty rate in the pre-betting and post-betting eras (with each line representing each referee). This visualization highlights both the direction and magnitude of change for each official.

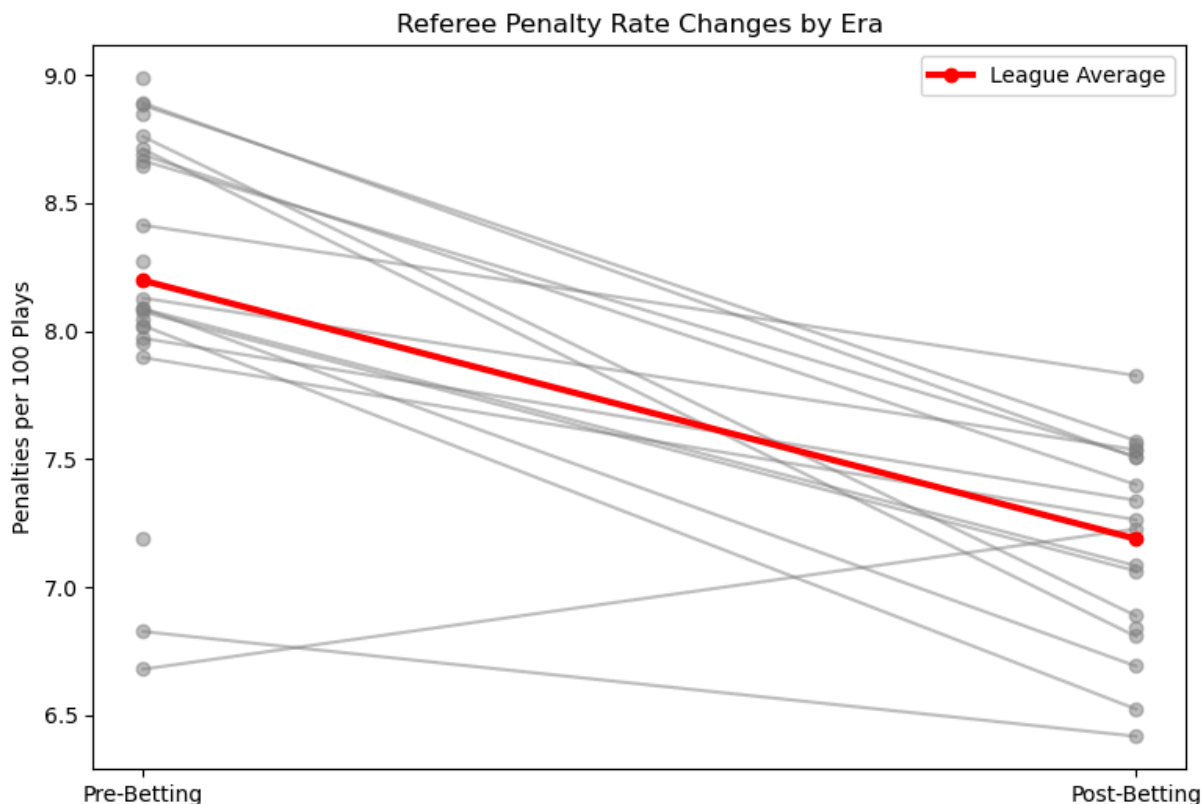
Figure 3.2.3

```
In [64]: # Slope plot for era change
# Pivot so we have pre and post-betting rates in separate columns
ref_era_pivot = ref_era_stats.pivot(index='referee', columns='era', values='penalti

# Plot slope plot
plt.figure(figsize=(9,6))
for i, row in ref_era_pivot.iterrows():
    plt.plot([0,1], [row['pre_betting'], row['post_betting']], marker='o', color='g

# Average line for each era
plt.plot([0,1], [ref_era_pivot['pre_betting'].mean(), ref_era_pivot['post_betting']
            marker='o', color='red', linewidth=3, label='League Average'])

plt.xticks([0,1], ['Pre-Betting', 'Post-Betting'])
plt.ylabel('Penalties per 100 Plays')
plt.title('Referee Penalty Rate Changes by Era')
plt.legend()
# plt.savefig("crew_penalty_rate_change.png", dpi=300, bbox_inches="tight")
plt.show()
```



The slope plot reveals a clear pattern: almost every referee called fewer penalties in the post-betting era compared to the pre-betting era. This downward shift is consistent across both high- and low-calling referees, suggesting a league-wide decrease in penalty frequency rather than changes isolated to a subset of officials. While the magnitude of change varies by referee, the direction is overwhelmingly downward. This visual complements the descriptive statistics, which show a decline in both the mean and median penalty rates as well as a reduction in variation across referees. Taken together, the slope plot and summary statistics indicate that referees' behavior became more consistent and slightly more restrained in the post-betting era. Importantly, while these patterns coincide with the expansion of legalized sports betting, other factors—such as evolving officiating guidelines, replay protocols, or league emphasis on game flow—could also contribute to this trend.

Crew-Team Behavior Across Eras

Finally, we compare referee–team interactions separately for the pre-betting and post-betting eras using two side-by-side heatmaps (Figure 3.3.4). These visuals allow us to identify whether the crew–team patterns observed earlier exhibit structural changes across eras.

Figure 3.2.4

```
In [65]: ## Same heatmap code with era (pre/post betting)

crew_team_plays_era = (
    all_team_plays
    .groupby(['era', 'referee', 'team'])
    .size()
```

```

        .reset_index(name = 'total_plays')
    )

    crew_team_penalties_era = (
        plays_games_subset[plays_games_subset['penalty'] == 1]
        .groupby(['era', 'referee', 'penalty_team'])
        .size()
        .reset_index(name = 'penalty_count')
    )

    crew_team_penalties_era = crew_team_penalties_era.rename(columns = {'penalty_team':

    crew_team_stats_era = crew_team_penalties_era.merge(
        crew_team_plays_era,
        on = ['era', 'referee', 'team'],
        how = 'left'
    )

    crew_team_stats_era['penalties_per_100'] = (crew_team_stats_era['penalty_count'] /

    crew_team_stats_era_filtered = crew_team_stats_era[crew_team_stats_era['referee'].i

    heatmap_pre = crew_team_stats_era[crew_team_stats_era['era'] == 'pre_betting'].pivo
        index = 'team',
        columns = 'referee',
        values = 'penalties_per_100'
    ).fillna(0)

    heatmap_post = crew_team_stats_era[crew_team_stats_era['era'] == 'post_betting'].pi
        index = 'team',
        columns = 'referee',
        values = 'penalties_per_100'
    ).fillna(0)

    # shared color scale
    vmin = min(heatmap_pre.min().min(), heatmap_post.min().min())
    vmax = max(heatmap_pre.max().max(), heatmap_post.max().max())

    # Plot side by side
    fig, axs = plt.subplots(1, 2, figsize=(24, 12))

    sns.heatmap(
        heatmap_pre,
        cmap = "viridis",
        ax = axs[0],
        vmin = vmin,
        vmax = vmax,
        linewidths = 0.4,
        linecolor = 'gray'
    )
    axs[0].set_title("Penalty Rate per 100 Plays – Pre-Betting (2015–2019)")
    axs[0].set_xlabel("Referee")
    axs[0].set_ylabel("Team")

    sns.heatmap(
        heatmap_post,

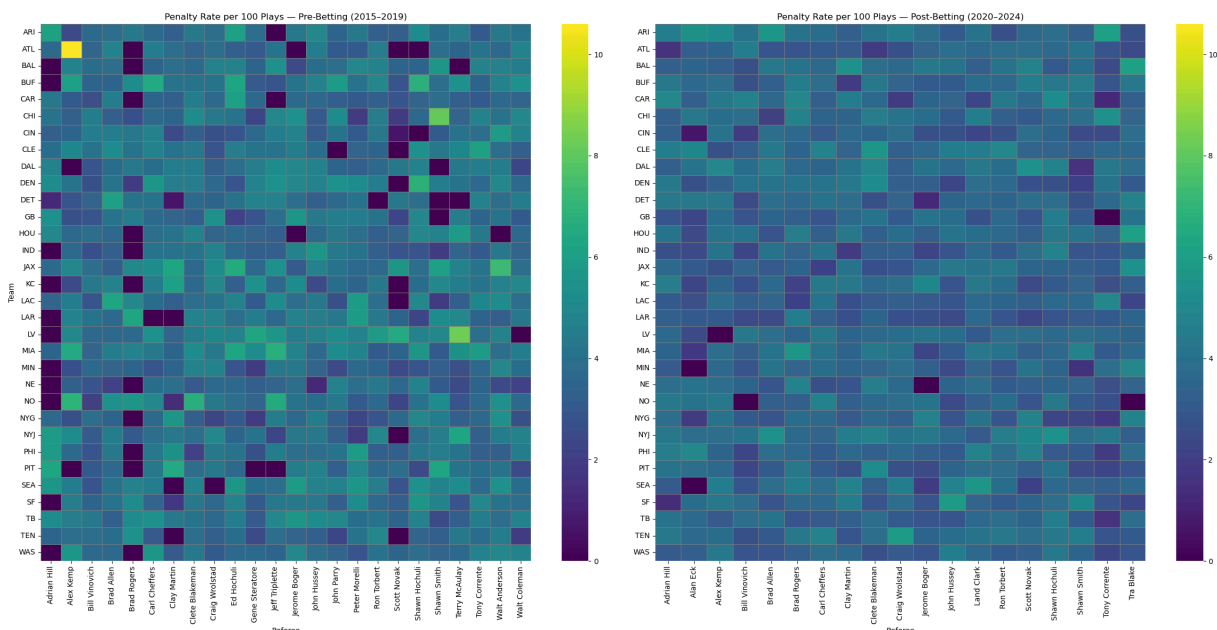
```

```

cmap = "viridis",
ax = axs[1],
vmin = vmin,
vmax = vmax,
linewidths = 0.4,
linecolor = 'gray'
)

axs[1].set_title("Penalty Rate per 100 Plays – Post-Betting (2020-2024)")
axs[1].set_xlabel("Referee")
axs[1].set_ylabel("")
plt.tight_layout()
# plt.savefig("ref_team_heatmap_era.png", dpi=300, bbox_inches="tight")
plt.show()

```



In the pre-betting era heatmap, there is a noticeable variation in color intensity across referees and teams, reflecting substantial differences in how often referees called penalties against certain teams. Some referees consistently called more penalties than others, and specific teams were subject to higher penalty rates with particular crews. By contrast, the post-betting era heatmap shows a more uniform color distribution across the matrix, indicating that penalty rates became more consistent both across referees and teams. The reduction in extreme values suggests that referees' behavior is more standardized and predictable in recent seasons, with fewer unusually high- or low-penalty combinations. This visual pattern aligns with the descriptive statistics and slope plots: the mean and median penalty rates decreased, and variation across referees shrank. Together, these results indicate that while individual differences still exist, the league has seen a general moderation and convergence in penalty-calling behavior in the post-betting era.

3.3 Analysis for Question 3: How have replay review outcomes, specifically reversals versus upheld calls, shifted between the pre-betting and post-betting era?

Finally, our group examined replay review reversal rates to determine if review outcomes have shifted between the pre-betting and post-betting eras. NFL replay reviews are often criticized due to inconsistent application, high-stakes, and debates over the clear evidence standard required to overturn rulings. Any shift in reversal tendencies after the 2019 season could signify sports betting influence on the results of replay reviews.

```
In [66]: # Check reversed vs upheld replay results
plays_games_subset['replay_or_challenge_result'].value_counts()
```

```
Out[66]: replay_or_challenge_result
reversed    1908
upheld      1838
denied        1
Name: count, dtype: int64
```

```
In [67]: # Figure 3.3.1

# Create reviews temporary df as copy of plays df, so we dont alter the original
reviews = plays_games_subset[plays_games_subset['replay_or_challenge'] == 1].copy()

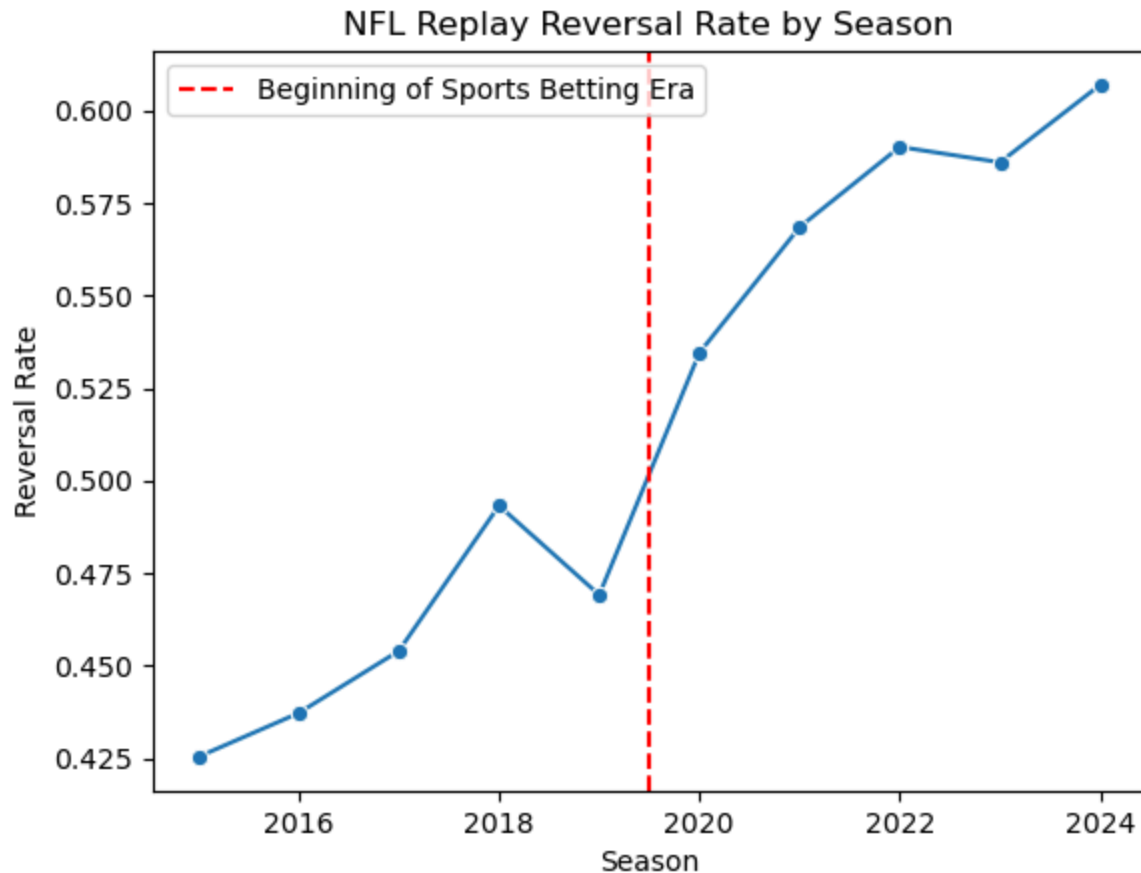
# Compute reversal flag
reviews['reversal'] = reviews['replay_or_challenge_result'].str.lower().str.contains('reversed')

# Calculate Reversal Rate by Season
reversal_rate = reviews.groupby('season')['reversal'].mean().reset_index()

# Lineplot
sns.lineplot(x='season', y='reversal', data=reversal_rate, marker='o')

# Add vertical line between 2019-2020 (widespread sports betting legalization)
plt.axvline(x=2019.5, linestyle='--', linewidth=1.5, label='Beginning of Sports Betting')
plt.legend()

plt.title("NFL Replay Reversal Rate by Season")
plt.ylabel("Reversal Rate")
plt.xlabel("Season")
plt.show()
```



As seen in Figure 3.3.1, the league's overall reversal rate shows a clear upward trend from 2015 to 2024, rising from 42.5% to over 60%. The upward trend begins in the pre-betting era; however, there is a dramatic increase in the trend after the 2019 season. In recent seasons, reviews have resulted in more overturned rulings. This upward shift aligns with the expansion of legalized sports betting; however, it does not definitively indicate causation. The shift could be due to several factors such as technological advances, evolving replay standards, or an increased willingness for teams to challenge borderline rulings.

```
In [68]: # Count total number of replay reviews by season
reviews_per_season = (
    reviews.groupby("season")
        .size()
        .reset_index(name="review_count")
)

reviews_per_season
```

Out[68]:

	season	review_count
0	2015	442
1	2016	366
2	2017	456
3	2018	369
4	2019	437
5	2020	391
6	2021	292
7	2022	305
8	2023	343
9	2024	346

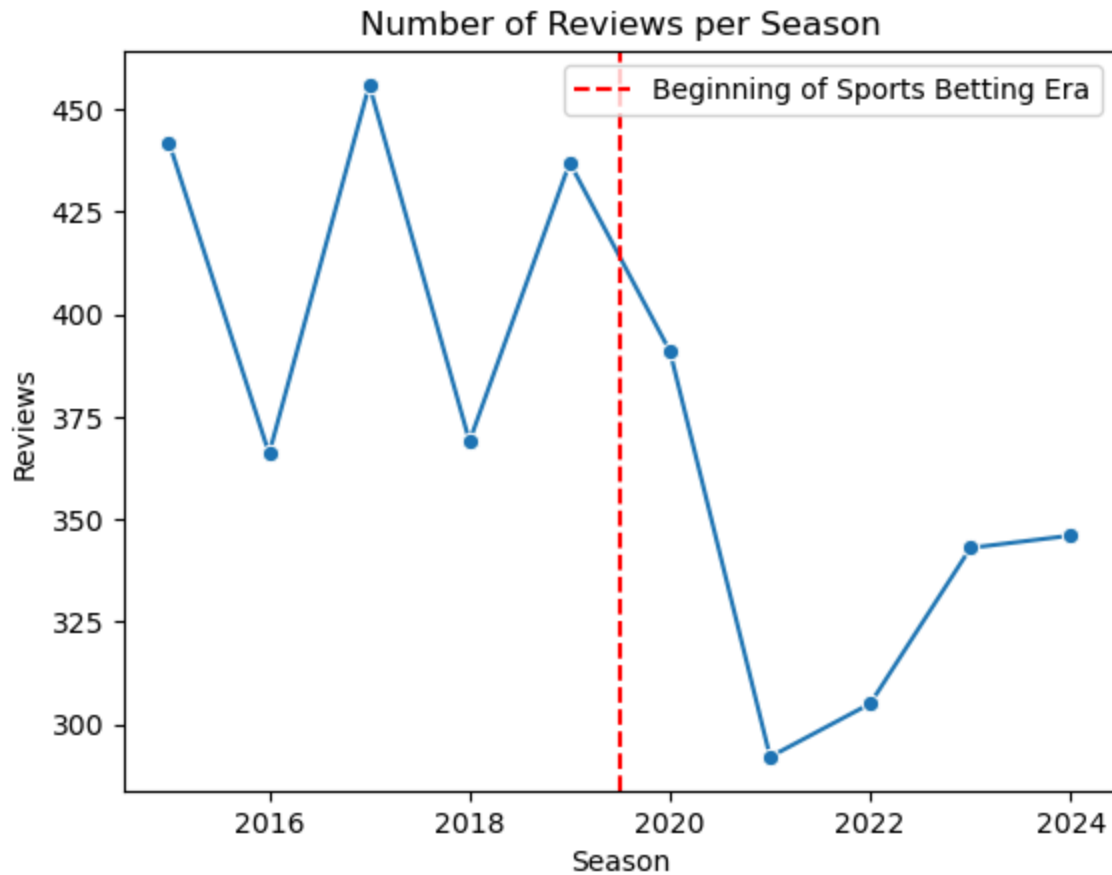
In [69]:

```
# Figure 3.3.2

# Lineplot of count of reviews per season
sns.lineplot(
    x="season",
    y="review_count",
    data=reviews_per_season,
    marker="o",
)

# Add vertical line between 2019-2020 (widespread sports betting legalization)
plt.axvline(x=2019.5, linestyle='--', linewidth=1.5, label='Beginning of Sports Bet
plt.legend()

plt.title("Number of Reviews per Season")
plt.ylabel("Reviews")
plt.xlabel("Season")
plt.show()
```



We examined the volume of replay reviews to determine if there was a correlation between the increase in reversed rulings and the total number of plays reviewed. Figure 3.3.2 shows that the number of reviews per season fluctuates, but it declines sharply in 2020 and partially rebounds from 2021 to 2024. It is important to note that as total reviews declined, reversal rates increased; thus, the increase in reversal rates is not due to teams challenging more rulings.

```
In [70]: play_type_counts = (  
    reviews.groupby("play_type")  
        .size()  
        .reset_index(name="num_reviews")  
        .sort_values("num_reviews", ascending=False)  
    )  
  
play_type_counts
```

Out[70]:

	play_type	num_reviews
4	pass	2712
6	run	799
3	no_play	115
5	punt	83
2	kickoff	34
0	extra_point	2
1	field_goal	2

```
In [71]: # Match Bar Chart and Interactive Plotly Chart to Default Plotly Colors to keep con
plotly_default_colors = ["#636EFA", "#EF553B", "#00CC96", "#AB63FA", "#FFA15A", "#19
play_types = sorted(play_type_counts['play_type'].dropna().unique())

# Map the colors to each play_type
color_map = {
    pt: plotly_default_colors[i % len(plotly_default_colors)]
    for i, pt in enumerate(play_types)
}
color_map
```

```
Out[71]: {'extra_point': '#636EFA',
'field_goal': '#EF553B',
'kickoff': '#00CC96',
'no_play': '#AB63FA',
'pass': '#FFA15A',
'punt': '#19D3F3',
'run': '#FF6692'}
```

Note: The color map above was used to ensure that the interactive line graph below of reversal rates over time grouped by play type and the bar graph below of the total amount of reviewed plays by play type use the same colors. It also ensures that the individual colors for the play type remain consistent.

```
In [72]: # Calculate reversal rate by play_type and season
play_type_season = (
    reviews.groupby(["season", "play_type"])["reversal"]
        .mean()
        .reset_index()
)
play_type_season.head()
```

Out[72]:

	season	play_type	reversal
0	2015	kickoff	0.40
1	2015	no_play	0.62
2	2015	pass	0.45
3	2015	punt	0.31
4	2015	run	0.36

```
In [73]: # Figure 3.3.3

# Use color map to ensure consistency
plotly_color_map = {
    pt: color_map[pt] for pt in play_types
}

# Plotly Lineplot (Interactive) of reversal rate by play_type and season
fig = px.line(
    play_type_season,
    x="season",
    y="reversal",
    color="play_type",
    color_discrete_map=plotly_color_map,
    markers=True,
    title="Reversal Rate by Play Type Over Time",
    labels={
        "season": "Season",
        "reversal": "Reversal Rate",
        "play_type": "Play Type"
    }
)

#Add Vertical Line to mark beginning of sports betting era
fig.add_vline(
    x=2019.5,
    line_width=2,
    line_dash="dash",
    line_color="red"
)

# Annotate the dashed vertical line and add arrow
fig.add_annotation(
    x=2019.5,
    y=max(play_type_season['reversal']),
    text="Beginning of Sports Betting Era",
    showarrow=True,
    arrowhead=1,
    standoff=5
)

# Add Legend title, ensure every season is displayed, and resize the figure
fig.update_layout(
    legend_title="Play Type",
```

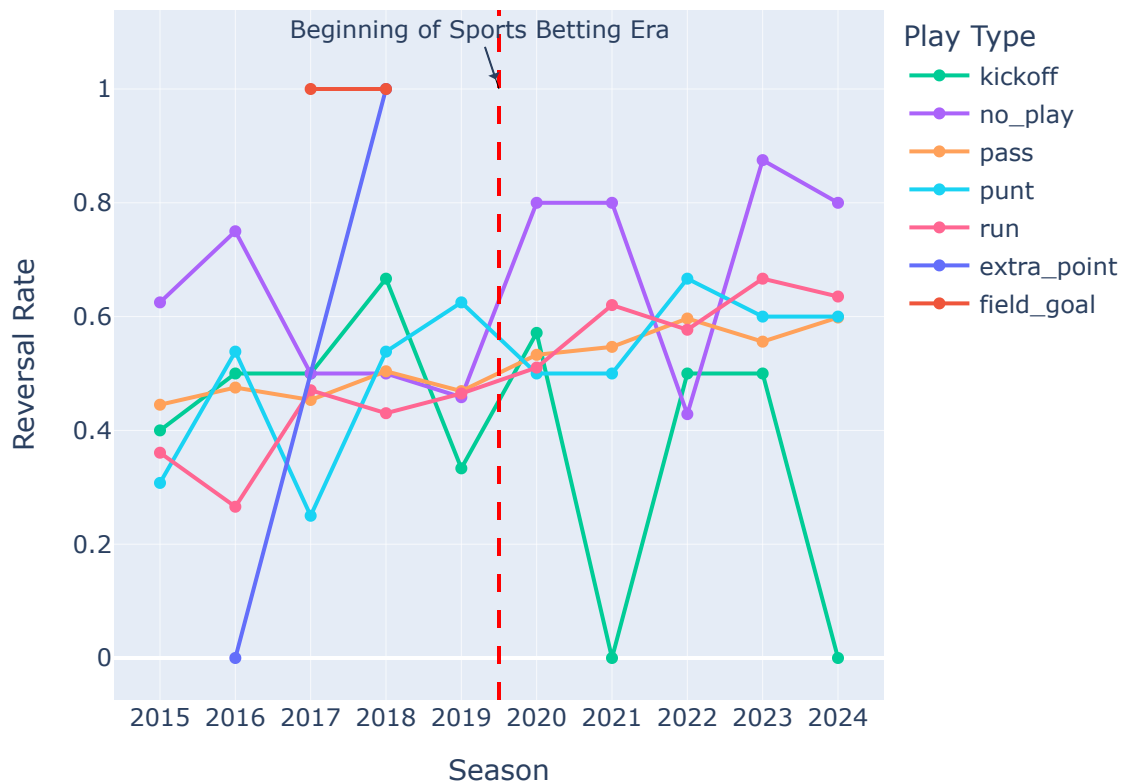
```

axis=dict(tickmode='linear'),
width=600
)

# Display the figure
fig.show()

```

Reversal Rate by Play Type Over Time



Next, we analyzed reversal rates and total reviews grouped by play type. In Figure 3.3.3, most play types show mostly noise, likely due to a small amount of these types of plays being reviewed every season. However, passing plays and running plays show steady upward trends in reversal rates.

```

In [74]: # Figure 3.3.4

# Bar Chart Showing Total Count of Replay Reviews for each play_type
plt.figure(figsize=(10,5))
sns.barplot(
    y="play_type",
    x="num_reviews",
    data=play_type_counts,
    hue="play_type",
    palette=color_map
)

```

```
plt.title("Number of Reviews by Play Type (2015-2024)")
plt.xlabel("Number of Reviews")
plt.ylabel("Play Type")
plt.show()
```

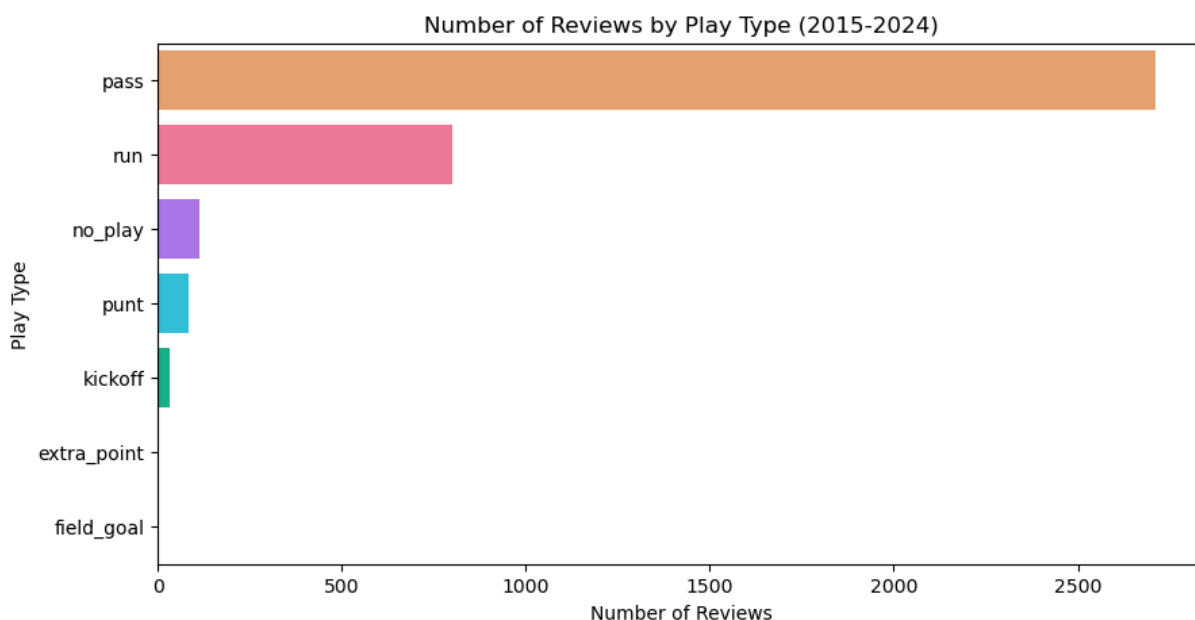


Figure 3.3.4 reveals that review counts are heavily concentrated in passing plays and running plays, which account for most of the replay reviews from 2015–2024. Thus, the behavior of reversals is largely dependent on the results of replay reviews of passing and running plays. Other categories such as no play, punt, kickoffs, and field goals appear so infrequently that they contribute minimally to league-wide patterns, and much of their trends are the result of noise.

```
In [75]: # New df getting the total reviews and the number of reversals grouped by referee
crew_stats = (
    reviews.groupby("referee")
        .agg(
            total_reviews=("reversal", "size"),
            reversals=("reversal", "sum")
        )
        .reset_index()
)

# Calculate reversal_rate
crew_stats["reversal_rate"] = crew_stats["reversals"] / crew_stats["total_reviews"]

# New df getting the number of seasons each referee has officiated
ref_seasons = (
    reviews.groupby("referee")["season"]
        .nunique()
        .reset_index(name="num_seasons")
)

# Merge dfs
```

```
crew_stats = crew_stats.merge(ref_seasons, on="referee", how="left")
crew_stats.head()
```

Out[75]:

	referee	total_reviews	reversals	reversal_rate	num_seasons
0	Adrian Hill	133	78	0.59	6
1	Alan Eck	45	26	0.58	2
2	Alex Kemp	138	68	0.49	7
3	Bill Vinovich	252	139	0.55	10
4	Brad Allen	230	107	0.47	10

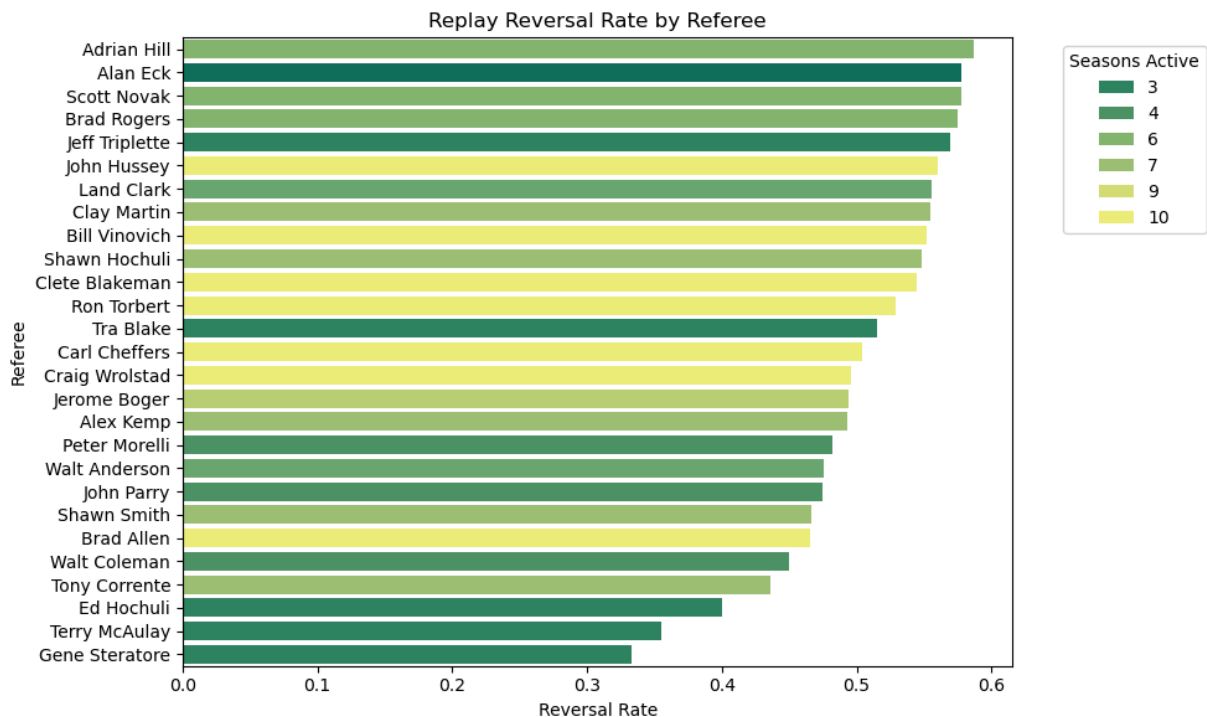
In [76]:

```
# Figure 3.3.5

# Barplot of Reversal Rate per ref with colors differentiating number of active sea
plt.figure(figsize=(10,6))

sns.barplot(
    data=crew_stats.sort_values("reversal_rate", ascending=False),
    y="referee",
    x="reversal_rate",
    hue="num_seasons",
    palette="summer"
)

plt.title("Replay Reversal Rate by Referee")
plt.xlabel("Reversal Rate")
plt.ylabel("Referee")
plt.legend(title="Seasons Active", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



Finally, Figure 3.3.5 highlights variation in individual referee reversal rates as well as the number of seasons each referee was active. It is apparent that there is a large variation in reversal rates across referees. The reversal rates range from 35% to 60% approximately. For the most part, referees with more experience (7-10 seasons) cluster near the middle of the distribution, while the less experienced referees (1-4 seasons) appear at the high and low ends of the distribution. This suggests that referees' reversal rates stabilize over time. The more extreme reversal rates for less experienced referees may be due to the unpredictability of small samples. Overall, there does not appear to be a consistent shift among referees after 2019, as referees with less than 6 years of experience are present at both the upper and lower extremes of the reversal rate distribution. This indicates that the differences across referees are likely independent of the sports betting timeline.

Our analysis indicates that replay reversal rates have increased notably in the sports betting era; however, this increase is part of a longer upward trajectory beginning before 2019. Replay review volume has seemingly decreased during this time, which suggests that officials are increasingly likely to overturn calls when reviews occur, but teams are less likely to challenge plays. Differences across play types and referees exist, but they do not reveal any clear structural changes that coincide directly with the expansion of legalized sports betting. Instead, the increase in overturned rulings appears to be due to other potential factors such as evolving replay standards, technological improvements, or rule changes.

4. Assumptions & Limitations

Throughout our analysis, we are assuming that the nflreadpy play-by-play and game metadata are complete for all seasons used in this report. We are also assuming that all

penalty and referee indicators are defined and recorded consistently, especially through any updates to the NFL rules and enforcement. While officiating crews may change slightly from season to season, we are also assuming that the listed referee or crew chief adequately represents the unit responsible for penalty and review decisions. Moreover, we assume that all NFL games are recorded with the same level of accuracy, no matter the location or teams playing. Finally, we noted that pre- and post-betting eras are defined as 2015-2019 and 2020-2024. As previously mentioned, the legalization of sports betting occurred after a 2018 Supreme Court ruling removed the federal prohibition on sports betting. After that announcement, only New Jersey and West Virginia allowed for sports betting to occur that same year. These two states have a minimal impact on the NFL betting scene, so we decided to wait until the beginning of the 2020 NFL season to begin the post-betting era. This marked a much larger turn towards the legalization of sports betting across the United States.

Our analysis is strictly descriptive. We are not using any statistical techniques that allow for causal relationships to be determined. We are simply extracting and comparing values to show any observational differences in the data. In addition, the NFL experiences lots of variation. There are uneven number of plays per team per season. There are also frequent rule changes to how certain penalties are called or enforced. Furthermore, officiating crews are not randomly assigned to games. Some crews may officiate more primetime, rivalry, or high-stakes games than others. These may consistently favor certain teams, matchups, or divisions over others. Teams also change their personnel, formations, and strategies over time. These shifts can influence various factors, including the types of penalties called or the number of replayable plays.

5. Conclusion

This analysis explores changes in penalty trends, officiating crew tendencies, and replay review outcomes between the pre-sports betting era (2015-2019) and the post-sports betting era (2020-2024).

First, an analysis of penalty trends shows a noticeable decline in frequency and yardage in the post-betting era. Teams committed fewer penalties per game, and this was accompanied by a decline in total penalty yardage per game. Importantly, the five most frequent penalties remained consistent across eras, and the reductions observed were small. This suggests that the decline in penalty frequency and yardage was not due to different penalties being called.

Second, an analysis of penalty frequencies by referee also shows a noticeable decline in penalties called. Interestingly, referees' penalty rates per team suggest that referees have gotten more consistent with their penalty calls from the pre-betting era to the post-betting era. There are many factors that dictate penalty rates, including NFL rule changes or more consistent referee training. This may not be the result of sports betting legalization.

Finally, an analysis of replay review outcomes shows that reversal rates increased notably between the pre-betting and post-betting eras but is part of an upward trend that began before 2019. It is important to note that there is a dramatic increase in the trend after the 2019 season. However, the number of replay reviews per season declined, which suggests that officials are more likely to overturn calls, but teams are less likely to challenge plays. Differences across play types and referees exist, but they do not directly correspond with legalized sports betting. Overall, these patterns suggest alternative explanations for officiating shifts, such as technological improvements or rule changes.

Our results show changes in NFL officiating patterns between the pre-betting and post-betting eras, but they are likely not the result of the widespread legalization of sports betting. The NFL is a very large organization that frequently changes its rules, regulations, and guidelines. There are a multitude of factors that may contribute to the decisions made by officiating crews that are not covered in this descriptive analysis.

Future work with more in-depth statistical analysis is needed to reveal any causal relationships between the legalization of sports betting and changes in NFL officiating patterns.

6. References

- <https://www.statnews.com/2025/11/11/sports-betting-apps-public-health-crisis/#:~:text=When%20the%20Supreme%20Court%20struck,men%20as%20its%20primary>
- <https://plotly.com/python/line-charts/>
- <https://matplotlib.org/stable/users/explain/colors/colormaps.html>
- <https://www.legalsportsreport.com/sports-betting-states/>



In []: